Program 9-13

```
// This program demonstrates a pointer to const parameter
#include <iostream>
using namespace std;

void displayValues(const int *, int);

int main()
{
    // Array sizes
    const int SIZE = 6;

    // Define an array of const ints.
    const int array1[SIZE] = { 1, 2, 3, 4, 5, 6 };

    // Define an array of nonconst ints.
    int array2[SIZE] = { 2, 4, 6, 8, 10, 12 };

    // Display the contents of the const array.
    displayValues(array1, SIZE);

    // Display the contents of the nonconst array.
    displayValues(array2, SIZE);
    return 0;
}

//************************************************
// The displayValues function uses a pointer to    *
// parameter to display the contents of an array.  *
//************************************************

void displayValues(const int *numbers, int size)
{
    // Display all the values.
    for (int count = 0; count < size; count++)
    {
         cout << *(numbers + count) << " ";
    }
    cout << endl;
}
```

Program Output

```
1 2 3 4 5 6
2 4 6 8 10 12
```



NOTE: When you are writing a function that uses a pointer parameter, and the function is not intended to change the data the parameter points to, it is always a good idea to make the parameter a pointer to `const`. Not only will this protect you from writing code in the function that accidentally changes the argument, but the function will be able to accept the addresses of both constant and nonconstant arguments.

Constant Pointers

In the previous section, we discussed pointers to `const`, that is, pointers that point to `const` data. You can also use the `const` key word to define a constant pointer. Here are the differences between a pointer to `const` and a `const` pointer:

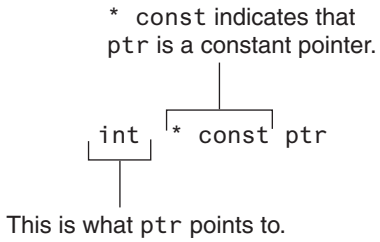
- A pointer to `const` points to a constant item. The data that the pointer points to cannot change, but the pointer itself can change.
- With a `const` pointer, it is the pointer itself that is constant. Once the pointer is initialized with an address, it cannot point to anything else.

The following code shows an example of a `const` pointer:

```
int value = 22;
int * const ptr = &value;
```

Notice in the definition of `ptr`, the word `const` appears after the asterisk. This means that `ptr` is a `const` pointer. This is illustrated in Figure 9-9. In the code, `ptr` is initialized with the address of the `value` variable. Because `ptr` is a constant pointer, a compiler error will result if we write code that makes `ptr` point to anything else. An error will not result, however, if we use `ptr` to change the contents of `value`. This is because `value` is not constant, and `ptr` is not a pointer to `const`.

Figure 9-9 `ptr` is a `const` pointer to `int`



Constant pointers must be initialized with a starting value, as shown in the previous example code. If a constant pointer is used as a function parameter, the parameter will be initialized with the address that is passed as an argument into it, and cannot be changed to point to anything else while the function is executing. Here is an example that attempts to violate this rule:

```
void setToZero(int * const ptr)
{
   ptr = 0; // ERROR!! Cannot change the contents of ptr.
}
```

This function's parameter, `ptr`, is a `const` pointer. It will not compile because we cannot have code in the function that changes the contents of `ptr`. However, `ptr` does not point to a `const`, so we can have code that changes the data that `ptr` points to. Here is an example of the function that will compile:

```
void setToZero(int * const ptr)
{
   *ptr = 0;
}
```

Although the parameter is const pointer, we can call the function multiple times with different arguments. The following code will successfully pass the addresses of x, y, and z to the setToZero function:

```
int x, y, z;
// Set x, y, and z to 0.
setToZero(&x);
setToZero(&y);
setToZero(&z);
```

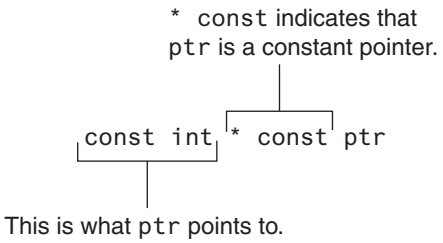
Constant Pointers to Constants

So far, when using const with pointers, we've seen pointers to constants and we've seen constant pointers. You can also have constant pointers to constants. For example, look at the following code:

```
int value = 22;
const int * const ptr = &value;
```

In this code, ptr is a const pointer to a const int. Notice the word const appears before int, indicating that ptr points to a const int, and it appears after the asterisk, indicating that ptr is a constant pointer. This is illustrated in Figure 9-10.

Figure 9-10 ptr is a const pointer to const int



In the code, ptr is initialized with the address of value. Because ptr is a const pointer, we cannot write code that makes ptr point to anything else. Because ptr is a pointer to const, we cannot use it to change the contents of value. The following code shows one more example of a const pointer to a const. This is another version of the displayValues function in Program 9-13.

```
void displayValues(const int * const numbers, int size)
{
   // Display all the values.
   for (int count = 0; count < size; count++)
   {
      cout << *(numbers + count) << " ";
   }
   cout << endl;
}
```

In this code, the parameter numbers is a const pointer to a const int. Although we can call the function with different arguments, the function itself cannot change what numbers points to, and it cannot use numbers to change the contents of an argument.

9.8 Dynamic Memory Allocation

CONCEPT: Variables may be created and destroyed while a program is running.

As long as you know how many variables you will need during the execution of a program, you can define those variables up front. For example, a program to calculate the area of a rectangle will need three variables: one for the rectangle's length, one for the rectangle's width, and one to hold the area. If you are writing a program to compute the payroll for 30 employees, you'll probably create an array of 30 elements to hold the amount of pay for each person.

But what about those times when you don't know how many variables you need? For instance, suppose you want to write a test-averaging program that will average any number of tests. Obviously the program would be very versatile, but how do you store the individual test scores in memory if you don't know how many variables to define? Quite simply, you allow the program to create its own variables "on the fly." This is called *dynamic memory allocation*, and is only possible through the use of pointers.

To dynamically allocate memory means that a program, while running, asks the computer to set aside a chunk of unused memory large enough to hold a variable of a specific data type. Let's say a program needs to create an integer variable. It will make a request to the computer that it allocate enough bytes to store an `int`. When the computer fills this request, it finds and sets aside a chunk of unused memory large enough for the variable. It then gives the program the starting address of the chunk of memory. The program can access the newly allocated memory only through its address, so a pointer is required to use those bytes.

The way a C++ program requests dynamically allocated memory is through the `new` operator. Assume a program has a pointer to an `int` defined as

```
int *iptr = nullptr;
```

Here is an example of how this pointer may be used with the `new` operator:

```
iptr = new int;
```

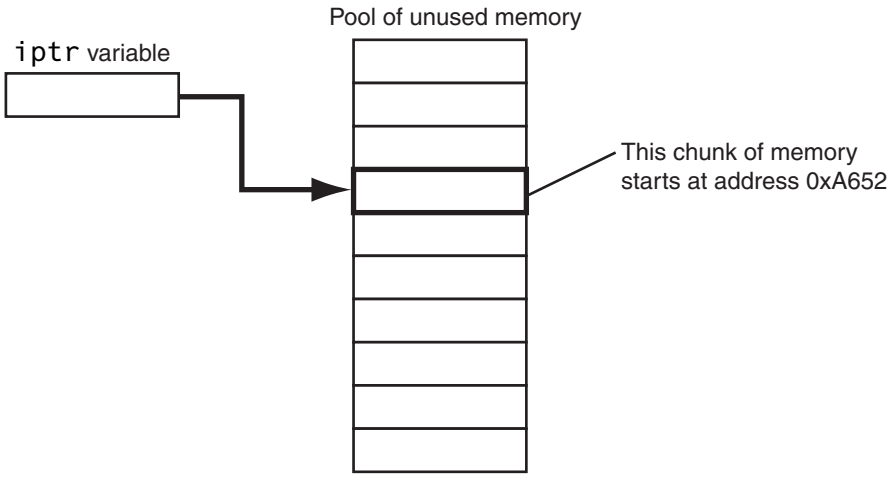
This statement is requesting that the computer allocate enough memory for a new `int` variable. The operand of the `new` operator is the data type of the variable being created. Once the statement executes, `iptr` will contain the address of the newly allocated memory. This is illustrated in Figure 9-11. A value may be stored in this new variable by dereferencing the pointer:

```
*iptr = 25;
```

Any other operation may be performed on the new variable by simply using the dereferenced pointer. Here are some example statements:

```
cout << *iptr;    // Display the contents of the new variable.
cin >> *iptr;     // Let the user input a value.
total += *iptr;   // Use the new variable in a computation.
```

Figure 9-11 `iptr` points to a chunk of dynamically allocated memory



Although the statements above illustrate the use of the `new` operator, there's little purpose in dynamically allocating a single variable. A more practical use of the `new` operator is to dynamically create an array. Here is an example of how a 100-element array of integers may be allocated:

```
iptr = new int[100];
```



VideoNote
Dynamically Allocating an Array

Once the array is created, the pointer may be used with subscript notation to access it. For instance, the following loop could be used to store the value 1 in each element:

```
for (int count = 0; count < 100; count++)
    iptr[count] = 1;
```

But what if there isn't enough free memory to accommodate the request? What if the program asks for a chunk large enough to hold a 100,000-element array of `floats`, and that much memory isn't available? When memory cannot be dynamically allocated, C++ throws an exception and terminates the program. *Throwing an exception* means the program signals that an error has occurred. You will learn more about exceptions in Chapter 16.

When a program is finished using a dynamically allocated chunk of memory, it should release it for future use. The `delete` operator is used to free memory that was allocated with `new`. Here is an example of how `delete` is used to free a single variable, pointed to by `iptr`:

```
delete iptr;
```

If `iptr` points to a dynamically allocated array, the `[]` symbol must be placed between `delete` and `iptr`:

```
delete [] iptr;
```

Failure to release dynamically allocated memory can cause a program to have a *memory leak*. For example, suppose the following `grabMemory` function is in a program:

```
void grabMemory()
{
   const int SIZE = 100;
   // Allocate space for a 100-element
   // array of integers.
   int *iptr = new int[SIZE];
   // The function ends without deleting the memory!
}
```

Notice the function dynamically allocates a 100-element `int` array and assigns its address to the `iptr` pointer variable. But then, the function ends without deleting the memory. Once the function has ended, the `iptr` pointer variable no longer exists. As a result, the program does not have a pointer to the dynamically allocated memory; therefore, the memory cannot be accessed or deleted. It will sit unused as long as the program is running.



WARNING! Only use pointers with `delete` that were previously used with `new`. If you use a pointer with `delete` that does not reference dynamically allocated memory, unexpected problems could result!

Program 9-14 demonstrates the use of `new` and `delete`. It asks for sales amounts for any number of days. The amounts are stored in a dynamically allocated array, then totaled and averaged.

Program 9-14

```
1  // This program totals and averages the sales amounts for any
2  // number of days. The amounts are stored in a dynamically
3  // allocated array.
4  #include <iostream>
5  #include <iomanip>
6  using namespace std;
7
8  int main()
9  {
10     double *sales = nullptr, // To dynamically allocate an array
11            total = 0.0,      // Accumulator
12            average;          // To hold average sales
13     int numDays,             // To hold the number of days of sales
14         count;               // Counter variable
15
16     // Get the number of days of sales.
17     cout << "How many days of sales amounts do you wish ";
18     cout << "to process? ";
19     cin >> numDays;
```

```
20
21      // Dynamically allocate an array large enough to hold
22      // that many days of sales amounts.
23      sales = new double[numDays];
24
25      // Get the sales amounts for each day.
26      cout << "Enter the sales amounts below.\n";
27      for (count = 0; count < numDays; count++)
28      {
29          cout << "Day " << (count + 1) << ": ";
30          cin >> sales[count];
31      }
32
33      // Calculate the total sales
34      for (count = 0; count < numDays; count++)
35      {
36          total += sales[count];
37      }
38
39      // Calculate the average sales per day
40      average = total / numDays;
41
42      // Display the results
43      cout << fixed << showpoint << setprecision(2);
44      cout << "\n\nTotal Sales: $" << total << endl;
45      cout << "Average Sales: $" << average << endl;
46
47      // Free dynamically allocated memory
48      delete [] sales;
49      sales = nullptr;    // Make sales a null pointer.
50
51      return 0;
52  }
```

Program Output with Example Input Shown in Bold

```
How many days of sales amounts do you wish to process? 5 Enter
Enter the sales amounts below.
Day 1: 898.63 Enter
Day 2: 652.32 Enter
Day 3: 741.85 Enter
Day 4: 852.96 Enter
Day 5: 921.37 Enter

Total Sales: $4067.13
Average Sales: $813.43
```

The statement in line 23 dynamically allocates memory for an array of `doubles`, using the value in `numDays` as the number of elements. The `new` operator returns the starting address of the chunk of memory, which is assigned to the `sales` pointer variable. The `sales` variable is then used throughout the program to store the sales amounts in the array and perform the necessary calculations. In line 48, the `delete` operator is used to free the allocated memory.

Notice in line 49, the value `nullptr` is assigned to the `sales` pointer. It is a good practice to set a pointer variable to `nullptr` after using `delete` on it. First, it prevents code from inadvertently using the pointer to access the area of memory that was freed. Second, it prevents errors from occurring if `delete` is accidentally called on the pointer again. The `delete` operator is designed to have no effect when used on a null pointer.

9.9 Returning Pointers from Functions

CONCEPT: Functions can return pointers, but you must be sure the item the pointer references still exists.

Like any other data type, functions may return pointers. For example, the following function locates the null terminator that appears at the end of a string (such as a string literal) and returns a pointer to it:

```
char *findNull(char *str)
{
   char *ptr = str;

   while (*ptr != '\0')
     ptr++;
   return ptr;
}
```

The `char *` return type in the function header indicates the function returns a pointer to a `char`.

```
char *findNull(char *str)
```

When writing functions that return pointers, you should take care not to create elusive bugs. For instance, see if you can determine what's wrong with the following function:

```
string *getFullName()
{
   string fullName[3];
   cout << "Enter your first name: ";
   getline(cin, fullName[0]);
   cout << "Enter your middle name: ";
   getline(cin, fullName[1]);
   cout << "Enter your last name: ";
   getline(cin, fullName[2]);
   return fullName;
}
```

The problem, of course, is that the function returns a pointer to an array that no longer exists. Because the `fullName` array is defined locally, it is destroyed when the function terminates. Attempting to use the pointer will result in erroneous and unpredictable results.

You should return a pointer from a function only if it is:

- A pointer to an item that was passed into the function as an argument.
- A pointer to a dynamically allocated chunk of memory.

For instance, the following function is acceptable:

```
string *getFullName(string fullName[])
{
   cout << "Enter your first name: ";
   getline(cin, fullName[0]);
   cout << "Enter your middle name: ";
   getline(cin, fullName[1]);
   cout << "Enter your last name: ";
   getline(cin, fullName[2]);
   return fullName;
}
```

This function accepts a pointer to the memory location where the user's input is to be stored. Because the pointer references a memory location that was valid prior to the function being called, it is safe to return a pointer to the same location. Here is another acceptable function:

```
string *getFullName()
{
   string *fullName = new string[3];

   cout << "Enter your first name: ";
   getline(cin, fullName[0]);
   cout << "Enter your middle name: ";
   getline(cin, fullName[1]);
   cout << "Enter your last name: ";
   getline(cin, fullName[2]);
   return fullName;
}
```

This function uses the `new` operator to allocate a section of memory. This memory will remain allocated until the `delete` operator is used or the program ends, so it's safe to return a pointer to it.

Program 9-15 shows another example. This program uses a function, `getRandomNumbers`, to get a pointer to an array of random numbers. The function accepts an integer argument that is the number of random numbers in the array. The function dynamically allocates an array, uses the system clock to seed the random number generator, populates the array with random values, then returns a pointer to the array.

Program 9-15

```
1  // This program demonstrates a function that returns
2  // a pointer.
3  #include <iostream>
4  #include <cstdlib>  // For rand and srand
5  #include <ctime>    // For the time function
6  using namespace std;
7
8  // Function prototype
9  int *getRandomNumbers(int);
10
```

(program continues)

Program 9-15 *(continued)*

```
11  int main()
12  {
13      int *numbers = nullptr;  // To point to the numbers
14
15      // Get an array of five random numbers.
16      numbers = getRandomNumbers(5);
17
18      // Display the numbers.
19      for (int count = 0; count < 5; count++)
20          cout << numbers[count] << endl;
21
22      // Free the memory.
23      delete [] numbers;
24      numbers = nullptr;
25      return 0;
26  }
27
28  //**************************************************
29  // The getRandomNumbers function returns a pointer  *
30  // to an array of random integers. The parameter    *
31  // indicates the number of numbers requested.       *
32  //**************************************************
33
34  int *getRandomNumbers(int num)
35  {
36      int *arr = nullptr;  // Array to hold the numbers
37
38      // Return a null pointer if num is zero or negative.
39      if (num <= 0)
40          return nullptr;
41
42      // Dynamically allocate the array.
43      arr = new int[num];
44
45      // Seed the random number generator by passing
46      // the return value of time(0) to srand.
47      srand( time(0) );
48
49      // Populate the array with random numbers.
50      for (int count = 0; count < num; count++)
51          arr[count] = rand();
52
53      // Return a pointer to the array.
54      return arr;
55  }
```

Program Output

```
2712
9656
24493
12483
7633
```



In the Spotlight:

Suppose you are developing a program that works with arrays of integers, and you find that you frequently need to duplicate the arrays. Rather than rewriting the array-duplicating code each time you need it, you decide to write a function that accepts an array and its size as arguments, creates a new array that is a copy of the argument array, and returns a pointer to the new array. The function will work as follows:

Accept an array and its size as arguments.
Dynamically allocate a new array that is the same size as the argument array.
Copy the elements of the argument array to the new array.
Return a pointer to the new array.

Program 9-16 demonstrates the function, which is named `duplicateArray`.

Program 9-16

```
1  // This program uses a function to duplicate
2  // an int array of any size.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype
7  int *duplicateArray(const int *, int);
8  void displayArray(const int[], int);
9
10 int main()
11 {
12     // Define constants for the array sizes.
13     const int SIZE1 = 5, SIZE2 = 7, SIZE3 = 10;
14
15     // Define three arrays of different sizes.
16     int array1[SIZE1] = { 100, 200, 300, 400, 500 };
17     int array2[SIZE2] = { 10, 20, 30, 40, 50, 60, 70 };
18     int array3[SIZE3] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
19
20     // Define three pointers for the duplicate arrays.
21     int *dup1 = nullptr, *dup2 = nullptr, *dup3 = nullptr;
22
23     // Duplicate the arrays.
24     dup1 = duplicateArray(array1, SIZE1);
25     dup2 = duplicateArray(array2, SIZE2);
26     dup3 = duplicateArray(array3, SIZE3);
27
28     // Display the original arrays.
29     cout << "Here are the original array contents:\n";
30     displayArray(array1, SIZE1);
31     displayArray(array2, SIZE2);
32     displayArray(array3, SIZE3);
33
34     // Display the new arrays.
```

(program continues)

Program 9-16 *(continued)*

```
35       cout << "\nHere are the duplicate arrays: \n";
36       displayArray(dup1, SIZE1);
37       displayArray(dup2, SIZE2);
38       displayArray(dup3, SIZE3);
39
40       // Free the dynamically allocated memory and
41       // set the pointers to 0.
42       delete [] dup1;
43       delete [] dup2;
44       delete [] dup3;
45       dup1 = nullptr;
46       dup2 = nullptr;
47       dup3 = nullptr;
48       return 0;
49    }
50    //*******************************************************
51    // The duplicateArray function accepts an int array    *
52    // and an int that indicates the array's size. The     *
53    // function creates a new array that is a duplicate    *
54    // of the argument array and returns a pointer to the  *
55    // new array. If an invalid size is passed, the        *
56    // function returns a null pointer.                    *
57    //*******************************************************
58
59    int *duplicateArray(const int *arr, int size)
60    {
61       int *newArray = nullptr;
62
63       // Validate the size. If 0 or a negative
64       // number was passed, return a null pointer.
65       if (size <= 0)
66          return nullptr;
67
68       // Allocate a new array.
69       newArray = new int[size];
70
71       // Copy the array's contents to the
72       // new array.
73       for (int index = 0; index < size; index++)
74          newArray[index] = arr[index];
75
76       // Return a pointer to the new array.
77       return newArray;
78    }
79
80    //************************************************
81    // The displayArray function accepts an int array *
82    // and its size as arguments and displays the     *
83    // contents of the array.                         *
84    //************************************************
85
```

```
86    void displayArray(const int arr[], int size)
87    {
88       for (int index = 0; index < size; index++)
89          cout << arr[index] << " ";
90       cout << endl;
91    }
```

Program Output

```
Here are the original array contents:
100 200 300 400 500
10 20 30 40 50 60 70
1 2 3 4 5 6 7 8 9 10
Here are the duplicate arrays:
100 200 300 400 500
10 20 30 40 50 60 70
1 2 3 4 5 6 7 8 9 10
```

The `duplicateArray` function appears in lines 59 and 78. The `if` statement in lines 65 and 66 validates that `size` contains a valid array size. If `size` is 0 or less, the function immediately returns `nullptr` to indicate that an invalid size was passed.

Line 69 allocates a new array and assigns its address to the `newArray` pointer. Then, the loop in lines 73 and 74 copies the elements of the `arr` parameter to the new array. Then, the `return` statement in line 77 returns a pointer to the new array.



Checkpoint

9.9 Assuming `arr` is an array of `int`s, will each of the following program segments display "True" or "False"?

A)
```
if (arr < &arr[1])
    cout << "True";
else
    cout << "False";
```

B)
```
if (&arr[4] < &arr[1])
    cout << "True";
else
    cout << "False";
```

C)
```
if (arr != &arr[2])
    cout << "True";
else
    cout << "False";
```

D)
```
if (arr != &arr[0])
    cout << "True";
else
    cout << "False";
```

9.10 Give an example of the proper way to call the following function:

```
void makeNegative(int *val)
{
   if (*val > 0)
```

```
    *val = -(*val);
}
```

9.11 Complete the following program skeleton. When finished, the program will ask the user for a length (in inches), then convert that value to centimeters, and display the result. You are to write the function `convert`. (*Note:* 1 inch = 2.54 cm. Do not modify function `main`.)

```
#include <iostream>
#include <iomanip>
using namespace std;

// Write your function prototype here.

int main()
{
   double measurement;

   cout << "Enter a length in inches, and I will convert\n";
   cout << "it to centimeters: ";
   cin >> measurement;
   convert(&measurement);
   cout << fixed << setprecision(4);
   cout << "Value in centimeters: " << measurement << endl;
   return 0;
}
//
// Write the function convert here.
//
```

9.12 Look at the following array definition:

```
const int numbers[SIZE] = { 18, 17, 12, 14 };
```

Suppose we want to pass the array to the function `processArray` in the following manner:

```
processArray(numbers, SIZE);
```

Which of the following function headers is the correct one for the `processArray` function?

A) `void processArray(const int *arr, int size)`
B) `void processArray(int * const arr, int size)`

9.13 Assume `ip` is a pointer to an `int`. Write a statement that will dynamically allocate an integer variable and store its address in `ip`. Write a statement that will free the memory allocated in the statement you wrote above.

9.14 Assume `ip` is a pointer to an `int`. Then, write a statement that will dynamically allocate an array of 500 integers and store its address in `ip`. Write a statement that will free the memory allocated in the statement you just wrote.

9.15 What is a null pointer?

9.16 Give an example of a function that correctly returns a pointer.

9.17 Give an example of a function that incorrectly returns a pointer.

9.10 Using Smart Pointers to Avoid Memory Leaks

CONCEPT: **C++ 11 introduces smart pointers, objects that work like pointers, but have the ability to automatically delete dynamically allocated memory that is no longer being used.**



In C++ 11, you can use *smart pointers* to dynamically allocate memory and not worry about deleting the memory when you are finished using it. A smart pointer automatically deletes a chunk of dynamically allocated memory when the memory is no longer being used. This helps to prevent memory leaks from occurring.

C++ 11 provides three types of smart pointer: `unique_ptr`, `shared_ptr`, and `weak_ptr`.

These smart pointers are described in Table 9-1.

Table 9-1 Smart Pointer Types

Smart Pointer Type	Description
`unique_ptr`	A `unique_ptr` is the sole owner of a piece of dynamically allocated memory. No two `unique_ptrs` can point to the same piece of memory. When a `unique_ptr` goes out of scope, it automatically deallocates the piece of memory that it points to.
`shared_ptr`	A `shared_ptr` can share ownership of a piece of dynamically allocated memory. Multiple pointers of the `shared_ptr` type can point to the same piece of memory. The memory is deallocated when the last `shared_ptr` that is pointing to it is destroyed.
`weak_ptr`	A `weak_ptr` does not own the memory it points to, and cannot be used to access the memory's contents. It is used in special situations where the memory pointed to by a `shared_ptr` must be referenced without increasing the number of `shared_ptrs` that own it.

To use any of the smart pointers, you must `#include` the `<memory>` header file with the following directive:

```
#include <memory>
```

In this book, we introduce `unique_ptr`. The syntax for defining a `unique_ptr` is somewhat different from the syntax used in defining a regular pointer. Here is an example:

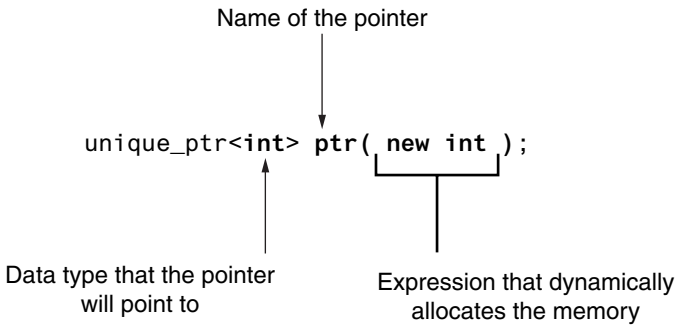
```
unique_ptr<int> ptr( new int );
```

This statement defines a `unique_ptr` named `ptr` that points to a dynamically allocated `int`. Here are some details about the statement:

- The notation `<int>` that appears immediately after `unique_ptr` indicates that the pointer can point to an `int`.
- The name of the pointer is `ptr`.
- The expression `new int` that appears inside the parentheses allocates a chunk of memory to hold an `int`. The address of the chunk of memory will be assigned to the `ptr` pointer.

Figure 9-12 shows the different parts of the definition statement.

Figure 9-12 Definition of a `unique_ptr`



Once you have defined a `unique_ptr`, you can use it in a similar way as a regular pointer. This is demonstrated in Program 9-17.

Program 9-17

```
1  // This program demonstrates a unique_ptr.
2  #include <iostream>
3  #include <memory>
4  using namespace std;
5
6  int main()
7  {
8       // Define a unique_ptr smart pointer, pointing
9       // to a dynamically allocated int.
10      unique_ptr<int> ptr( new int );
11
12      // Assign 99 to the dynamically allocated int.
13      *ptr = 99;
14
15      // Display the value of the dynamically allocated int.
16      cout << *ptr << endl;
17      return 0;
18 }
```

Program Output

```
99
```

In line 3, we have a `#include` directive for the `<memory>` header file. Line 10 defines a `unique_ptr` named `ptr`, pointing to a dynamically allocated `int`. Line 13 assigns the value 99 to the dynamically allocated `int`. Notice the indirection operator (`*`) is used with the `unique_ptr`, just as if it were a regular pointer. Line 16 displays the value stored in the dynamically allocated `int`, once again using the indirection operator with the `unique_ptr`. Notice there is no `delete` statement to free the dynamically allocated memory. It is unnecessary to delete the memory, because the smart pointer will automatically delete it as the function comes to an end.

Program 9-17 demonstrates a `unique_ptr`, but it isn't very practical. Dynamically allocating an array is more useful than allocating a single integer. The following code shows an example of how to use a `unique_ptr` to dynamically allocate an array of integers:

```
const int SIZE = 100;
unique_ptr<int[]> ptr( new int[SIZE] );
```

The first statement defines an `int` constant named `SIZE`, set to the value 100. The second statement defines a `unique_ptr` named `ptr` that points to a dynamically allocated array of 100 `int`s. Notice the following things about the definition statement:

- Following `unique_ptr`, the notation `<int[]>` indicates that the pointer will point to an array of `int`s.
- The expression inside the parentheses, `new int[SIZE]`, allocates an array of `int`s.

The address of the dynamically allocated array of `int`s will be assigned to the `ptr` pointer. After the definition statement, you can use the `[]` operator with subscripts to access the array elements. Here is an example:

```
ptr[0] = 99;
cout << ptr[0] << endl;
```

The first statement assigns the value 99 to `ptr[0]`, and the second statement displays the value of `ptr[0]`. Program 9-18 gives a more complete demonstration.

Program 9-18

```
1  // This program demonstrates a unique_ptr pointing
2  // to a dynamically allocated array of integers.
3  #include <iostream>
4  #include <memory>
5  using namespace std;
6
7  int main()
8  {
9      int max; // Max size of the array
10
11     // Get the number of values to store.
12     cout << "How many numbers do you want to enter? ";
13     cin >> max;
14
15     // Define a unique_ptr smart pointer, pointing
16     // to a dynamically allocated array of ints.
17     unique_ptr<int[]> ptr( new int[max]);
18
19     // Get values for the array.
20     for (int index = 0; index < max; index++)
21     {
22         cout << "Enter an integer number: ";
23         cin >> ptr[index];
24     }
25
```

(program continues)

Program 9-18 *(continued)*

```
26       // Display the values in the array.
27       cout << "Here are the values you entered:\n";
28       for (int index = 0; index < max; index++)
29           cout << ptr[index] << endl;
30
31       return 0;
32   }
```

Program Output with Example Input Shown in Bold

```
How many numbers do you want to enter? 5 Enter
Enter an integer number: 1 Enter
Enter an integer number: 2 Enter
Enter an integer number: 3 Enter
Enter an integer number: 4 Enter
Enter an integer number: 5 Enter
Here are the values you entered:
1
2
3
4
5
```

You can define an uninitialized `unique_ptr`, then assign it a value in a later statement. Here is an example:

```
unique_ptr<int> ptr;
ptr = unique_ptr<int>(new int);
```

Smart pointers support the same `*` and `->` dereferencing operators as regular pointers, but smart pointers do not support pointer arithmetic. The following code will result in a compile-time error:

```
unique_ptr<int[]> ptr( new int[max]);
ptr++;    // ERROR!
```

9.11 Focus on Problem Solving and Program Design: A Case Study

CONCEPT: **This case study demonstrates how an array of pointers can be used to display the contents of a second array in sorted order, without sorting the second array.**

The United Cause, a charitable relief agency, solicits donations from businesses. The local United Cause office received the following donations from the employees of CK Graphics, Inc.:

```
$5, $100, $5, $25, $10, $5, $25, $5, $5, $100, $10, $15, $10, $5, $10
```

The donations were received in the order they appear. The United Cause manager has asked you to write a program that displays the donations in ascending order, as well as in their original order.

Variables

Table 9-2 shows the major variables needed.

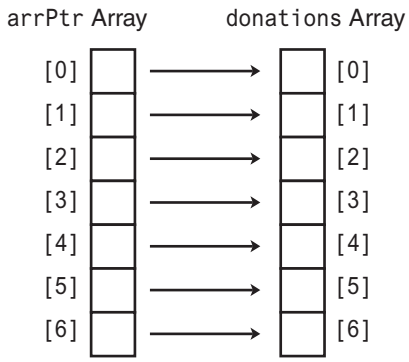
Table 9-2 Variables

Variable	Description
`NUM_DONATIONS`	A constant integer initialized with the number of donations received from CK Graphics, Inc. This value will be used in the definition of the program's arrays.
`donations`	An array of integers containing the donation amounts.
`arrPtr`	An array of pointers to integers. This array has the same number of elements as the `donations` array. Each element of `arrPtr` will be initialized to point to an element of the `donations` array.

Programming Strategy

In this program, the `donations` array will contain the donations in the order they were received. The elements of the `arrPtr` array are pointers to integers. They will point to the elements of the `donations` array, as illustrated in Figure 9-13.

Figure 9-13 Relationship between the arrays

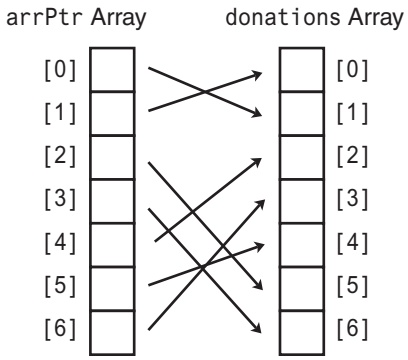


The `arrPtr` array will initially be set up to point to the elements of the `donations` array in their natural order. In other words, `arrPtr[0]` will point to `donations[0]`, `arrPtr[1]` will point to `donations[1]`, and so forth. In that arrangement, the following statement would cause the contents of `donations[5]` to be displayed:

```
cout << *(arrPtr[5]) << endl;
```

After the `arrPtr` array is sorted, however, `arrPtr[0]` will point to the smallest element of `donations`, `arrPtr[1]` will point to the next-to-smallest element of `donations`, and so forth. This is illustrated in Figure 9-14.

Figure 9-14 Accessing the donations array elements in sorted order



This technique gives us access to the elements of the `donations` array in a sorted order without actually disturbing the contents of the `donations` array itself.

Functions

The program will consist of the functions listed in Table 9-3.

Table 9-3 Functions

Function	Description
`main`	The program's `main` function. It calls the program's other functions.
`arrSelectSort`	Performs an ascending order selection sort on its parameter, `arr`, which is an array of pointers. Each element of `arr` points to an element of a second array. After the sort, `arr` will point to the elements of the second array in ascending order.
`showArray`	Displays the contents of its parameter, `arr`, which is an array of integers. This function is used to display the donations in their original order.
`showArrPtr`	Accepts an array of pointers to integers as an argument. Displays the contents of what each element of the array points to. This function is used to display the contents of the `donations` array in sorted order.

Function `main`

In addition to containing the variable definitions, function `main` sets up the `arrPtr` array to point to the elements of the `donations` array. Then, the function `arrSelectSort` is called to sort the elements of `arrPtr`. Last, the functions `showArrPtr` and `showArray` are called to display the donations. Here is the pseudocode for `main`'s executable statements:

```
For count is set to the values 0 through the number of donations
    Set arrPtr[count] to the address of donations[count].
End For
Call arrSelectSort
Call showArrPtr
Call showArray
```

The arrSelectSort Function

The arrSelectSort function is a modified version of the selection sort algorithm shown in Chapter 8. The only difference is that arr is now an array of pointers. Instead of sorting on the contents of arr's elements, arr is sorted on the contents of what its elements point to. Here is the pseudocode:

```
For startScan is set to the values 0 up to (but not including) the
        next-to-last subscript in arr
  Set index variable to startScan
  Set minIndex variable to startScan
  Set minElem pointer to arr[startScan]
  For index variable is set to the values from (startScan + 1) through
              the last subscript in arr
     If *(arr[index]) is less than *minElem
        Set minElem to arr[index]
        Set minIndex to index
     End If
  End For
  Set arr[minIndex] to arr[startScan]
  Set arr[startScan] to minElem
End For
```

The showArrPtr Function

The showArrPtr function accepts an array of pointers as its argument. It displays the values pointed to by the elements of the array. Here is its pseudocode:

```
For every element in the arr
   Dereference the element and display what it points to
End For.
```

The showArray Function

The showArray function simply displays the contents of arr sequentially. Here is its pseudocode:

```
For every element in arr
   Display the element's contents
End For.
```

The Entire Program

Program 9-19 shows the entire program's source code.

Program 9-19

```
1  // This program shows the donations made to the United Cause
2  // by the employees of CK Graphics, Inc. It displays
3  // the donations in order from lowest to highest
```

(program continues)

Program 9-19 *(continued)*

```
// and in the original order they were received.
#include <iostream>
using namespace std;

// Function prototypes
void arrSelectSort(int *[], int);
void showArray(const int [], int);
void showArrPtr(int *[], int);

int main()
{
    const int NUM_DONATIONS = 15;    // Number of donations

    // An array containing the donation amounts.
    int donations[NUM_DONATIONS] = { 5,  100, 5,  25, 10,
                                     5,  25,  5,  5,  100,
                                     10, 15,  10, 5,  10 };

    // An array of pointers to int.
     int *arrPtr[NUM_DONATIONS] = { nullptr, nullptr, nullptr, nullptr, nullptr,
                                    nullptr, nullptr, nullptr, nullptr, nullptr,
                                    nullptr, nullptr, nullptr, nullptr, nullptr};

    // Each element of arrPtr is a pointer to int. Make each
    // element point to an element in the donations array.
    for (int count = 0; count < NUM_DONATIONS; count++)
        arrPtr[count] = &donations[count];

    // Sort the elements of the array of pointers.
    arrSelectSort(arrPtr, NUM_DONATIONS);

    // Display the donations using the array of pointers. This
    // will display them in sorted order.
    cout << "The donations, sorted in ascending order, are: \n";
    showArrPtr(arrPtr, NUM_DONATIONS);

    // Display the donations in their original order.
    cout << "The donations, in their original order, are: \n";
    showArray(donations, NUM_DONATIONS);
    return 0;
}

//***************************************************************
// Definition of function arrSelectSort.                        *
// This function performs an ascending order selection sort on  *
// arr, which is an array of pointers. Each element of array    *
// points to an element of a second array. After the sort,      *
// arr will point to the elements of the second array in        *
// ascending order.                                             *
//***************************************************************

```

```
55  void arrSelectSort(int *arr[], int size)
56  {
57       int startScan, minIndex;
58       int *minElem;
59
60       for (startScan = 0; startScan < (size - 1); startScan++)
61       {
62            minIndex = startScan;
63            minElem = arr[startScan];
64            for(int index = startScan + 1; index < size; index++)
65            {
66                 if (*(arr[index]) < *minElem)
67                 {
68                      minElem = arr[index];
69                      minIndex = index;
70                 }
71            }
72            arr[minIndex] = arr[startScan];
73            arr[startScan] = minElem;
74       }
75  }
76
77  //********************************************************
78  // Definition of function showArray.                     *
79  // This function displays the contents of arr. size is the *
80  // number of elements.                                   *
81  //********************************************************
82
83  void showArray(const int arr[], int size)
84  {
85       for (int count = 0; count < size; count++)
86            cout << arr[count] << " ";
87       cout << endl;
88  }
89
90  //************************************************************
91  // Definition of function showArrPtr.                        *
92  // This function displays the contents of the array pointed to *
93  // by arr. size is the number of elements.                   *
94  //************************************************************
95
96  void showArrPtr(int *arr[], int size)
97  {
98       for (int count = 0; count < size; count++)
99            cout << *(arr[count]) << " ";
100      cout << endl;
101 }
```

Program Output

```
The donations, sorted in ascending order, are:
5 5 5 5 5 5 10 10 10 10 15 25 25 100 100
The donations, in their original order, are:
5 100 5 25 10 5 25 5 5 100 10 15 10 5 10
```

Review Questions and Exercises

Short Answer

1. What does the indirection operator do?
2. Look at the following code.

   ```
   int x = 7;
   int *iptr = &x;
   ```

   What will be displayed if you send the expression `*iptr` to `cout`? What happens if you send the expression `ptr` to `cout`?
3. So far you have learned three different uses for the * operator. What are they?
4. What math operations are allowed on pointers?
5. Assuming `ptr` is a pointer to an `int`, what happens when you add 4 to `ptr`?
6. Look at the following array definition.

   ```
   int numbers[] = { 2, 4, 6, 8, 10 };
   ```

   What will the following statement display?

   ```
   cout << *(numbers + 3) << endl;
   ```
7. What is the purpose of the `new` operator?
8. What happens when a program uses the `new` operator to allocate a block of memory, but the amount of requested memory isn't available? How do programs written with older compilers handle this?
9. What is the purpose of the `delete` operator?
10. Under what circumstances can you successfully return a pointer from a function?
11. What is the difference between a pointer to a constant and a constant pointer?
12. What are two advantages of declaring a pointer parameter as a constant pointer?

Fill-in-the-Blank

13. Each byte in memory is assigned a unique _____.
14. The statement `ptr = &num` assigns the _____ of `num` to the pointer `ptr`.
15. The statement cout `<< *(&num)` prints the _____ of the variable `num`.
16. A(n) _____ can hold the address of an integer variable.
17. Array names can be used as _____, and vice versa.
18. Creating variables while a program is running is called _____.
19. The _____ operator is used to dynamically allocate memory.
20. Under older compilers, if the `new` operator cannot allocate the amount of memory requested, it returns _____.
21. A pointer that contains the address 0 is called a(n) _____ pointer.
22. The _____ operator is used to release the amount of memory that was dynamically created using the _____ operator.
23. You should only use pointers with `delete` that were previously used with _____.

Algorithm Workbench

24. Look at the following code:

```
double value = 29.7;
double *ptr = &value;
```

Write a `cout` statement that uses the `ptr` variable to display the contents of the `value` variable.

25. Look at the following function call.

```
iptr = someFunc ( x, &y);
```

If x and y are integers and `iptr` is a pointer to an integer, write the function prototype.

26. Write code to dynamically allocate an integer array, allow the user to input the array elements, find their mean, and finally release the memory used by the array.
27. Write a single statement that displays the value pointed by a pointer `xptr`, and then increments the pointer `xptr`.
28. Look at the following function definition:

```
void assign ( int *numbers, int size )
{
   for ( int i = 0 ; i < size ; i++ )
        cin >> numbers[i];
}
```

Write code showing how you can call this function. Also, rewrite the `cin` statement to show an alternative way to input elements in the array.

29. Write the definition of `ptr`, a pointer to a constant `int`.
30. Write code to display a two-dimensional array `arr[5][10]`, using pointer notation.

True or False

31. T F Each byte of memory is assigned a unique address.
32. T F The * operator is used to get the address of a variable.
33. T F A pointer variable can be incremented and decremented.
34. T F The & symbol is called the indirection operator.
35. T F The statement const double `*dptr` indicates that `dptr` is a constant pointer.
36. T F When the indirection operator is used with a pointer variable, you are actually working with the value the pointer is pointing to.
37. T F Array names cannot be dereferenced with the indirection operator.
38. T F When you add a value to a pointer, you are actually adding that number times the size of the data type referenced by the pointer.
39. T F A constant pointer implies a pointer to a constant variable.
40. T F You can change the address that an array name points to.
41. T F Any mathematical operation, including multiplication and division, may be performed on a pointer.
42. T F Once a constant pointer is initialized with an address, it cannot point to anything else.
43. T F Though a function can accept pointer variables as arguments, a function can never return a pointer.

44. T F The new operator dynamically allocates memory.
45. T F A pointer variable that has not been initialized is called a null pointer.
46. T F The address 0 is generally considered unusable.
47. T F A smart pointer can automatically delete a dynamically allocated memory when it is not in use anymore, without the need of the delete operator to be explicitly called to do so.

Find the Error

Each of the following definitions and program segments has errors. Locate as many as you can.

48.
```
double *dptr = *nullptr;
```

49.
```
int x, *ptr = nullptr;
&x = ptr;
```

50.
```
int ptr, value=9;
ptr = &value;
```

51.
```
int x, *ptr = nullptr;
ptr = &x;
ptr = 100; // Store 100 in x
cout << x << endl;
```

52.
```
float numbers[] = { 2.3, 3.4, 4.5, 5.6, 6.7};
cout << "The second element in the list of numbers is :";
cout << (*numbers++) << endl;
```

53.
```
int values[20], *iptr = nullptr;
iptr = values;
iptr *= 2;
```

54.
```
float level;
int fptr = &level;
```

55.
```
float values[6] = {1,2,3,4,5,6};
cout << *(values + 1.5) << endl;
```

56.
```
void doubleVal(int val)
{
   *val *= 2;
}
```

57.
```
int *pint = nullptr;
new pint;
```

58.
```
int *pint = nullptr;
pint = new int;
if (pint == nullptr)
    *pint = 100;
else
    cout << "Memory allocation error\n";
```

59.
```
int *pint = nullptr;
pint = new int[100]; // Allocate memory
    .
    .
```
Code that processes the array.
```
    .
    .
delete pint; // Free memory
```

60.
```
int *getNum()
{
    int wholeNum;
    cout << "Enter a number: ";
    cin >> wholeNum;
    return &wholeNum;
}
```

61.
```
const int arr[] = { 1, 2, 3 };
int *ptr = arr;
```

62.
```
void doSomething(int * const ptr)
{
    int localArray[] = { 1, 2, 3 };
    ptr = localArray;
}
```

Programming Challenges

1. **Address of Elements in an Array**

   Write a program to initialize an array of integers with 10 numbers. The program should then display the number of bytes occupied by the array. Also display each element as well as the address in which it is stored.

2. **Test Scores #1**

   Write a program that dynamically allocates an array large enough to hold a user-defined number of test scores. Once all the scores are entered, the array should be passed to a function that sorts them in ascending order. Another function should be called that calculates the average score. The program should display the sorted list of scores and averages with appropriate headings. Use pointer notation rather than array notation whenever possible.

   Input Validation: Do not accept negative numbers for test scores.

3. **Drop Lowest Score**

   Modify Problem 2 above so the lowest test score is dropped. This score should not be included in the calculation of the average.

4. **Test Scores #2**

   Modify the program of Programming Challenge 2 (Test Scores #1) to allow the user to enter name–score pairs. For each student taking a test, the user types the student's name followed by the student's integer test score. Modify the sorting function so it takes an array holding the student names, and an array holding the student test scores. When the sorted list of scores is displayed, each student's name should be displayed along with his or her score. In stepping through the arrays, use pointers rather than array subscripts.

5. **Pointer Rewrite**



VideoNote
Solving the Pointer Rewrite Problem

   The following function uses reference variables as parameters. Rewrite the function so it uses pointers instead of reference variables, then demonstrate the function in a complete program.

```
int doSomething(int &x, int &y)
{
     int temp = x;
     x = y * 10;
     y = temp * 10;
     return x + y;
}
```

6. **Case Study Modification #1**

   Modify Program 9-19 (the United Cause case study program) so it can be used with any set of donations. The program should dynamically allocate the `donations` array and ask the user to input its values.

7. **Case Study Modification #2**

   Modify Program 9-19 (the United Cause case study program) so the `arrptr` array is sorted in descending order instead of ascending order.

8. **Mode Function**

   In statistics, the *mode* of a set of values is the value that occurs most often or with the greatest frequency. Write a function that accepts as arguments the following:

   A) An array of integers

   B) An integer that indicates the number of elements in the array

   The function should determine the mode of the array. That is, it should determine which value in the array occurs most often. The mode is the value the function should return. If the array has no mode (none of the values occur more than once), the function should return –1. (Assume the array will always contain nonnegative values.)

   Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.

9. **Median Function**

   In statistics, when a set of values is sorted in ascending or descending order, its *median* is the middle value. If the set contains an even number of values, the median is the

mean, or average, of the two middle values. Write a function that accepts as arguments the following:

A) An array of integers

B) An integer that indicates the number of elements in the array

The function should determine the median of the array. This value should be returned as a `double`. (Assume the values in the array are already sorted.)

Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.

10. Array Splitter

Write a function that accepts an `int` array and the array's size as arguments. The function should return a new array consisting of the values at even positions in the original array, that is, elements 0, 2, 4, 6, 8, and so on. The function should return a pointer to the new array. Demonstrate the function in a complete program.

11. Array Joiner

Write a function that accepts two `int` arrays and their sizes as arguments. The function should return a new array consisting of the values of both arrays. The elements of the second array should be joined after the elements of the first array. The function should return a pointer to the new array. Demonstrate the function in a complete program.

12. Array Filter

Write a function that accepts three arguments: an `int` array, its size, and an `int` variable. The function should return a new array consisting of only those values in the original array that are a multiple of the third argument. For example, if the third argument is 7, only those values from the original array that are multiples of 7 should be in the new array. First calculate the size of the new array and then copy the correct values. The function should return a pointer to the new array.

13. Movie Statistics

Write a program that can be used to gather statistical data about the number of movies college students see in a month. The program should perform the following steps:

A) Ask the user how many students were surveyed. An array of integers with this many elements should then be dynamically allocated.

B) Allow the user to enter the number of movies each student saw into the array.

C) Calculate and display the average, median, and mode of the values entered. (Use the functions you wrote in Programming Challenges 8 and 9 to calculate the median and mode.)

Input Validation: Do not accept negative numbers for input.

This page intentionally left blank

CHAPTER 10

Characters, C-Strings, and More about the `string` Class

TOPICS

10.1 Character Testing
10.2 Character Case Conversion
10.3 C-Strings
10.4 Library Functions for Working with C-Strings
10.5 String/Numeric Conversion Functions
10.6 Focus on Software Engineering: Writing Your Own C-String-Handling Functions
10.7 More about the C++ `string` Class
10.8 Focus on Problem Solving and Program Design: A Case Study

10.1 Character Testing

CONCEPT: The C++ library provides several functions for testing characters. To use these functions you must include the <cctype> header file.

The C++ library provides several functions that allow you to test the value of a character. These functions test a single `char` argument and return either `true` or `false`.* For example, the following program segment uses the `isupper` function to determine whether the character passed as an argument is an uppercase letter. If it is, the function returns `true`. Otherwise, it returns `false`.

```
char letter = 'a';
if (isupper(letter))
   cout << "Letter is uppercase.\n";
else
   cout << "Letter is lowercase.\n";
```

Because the variable `letter`, in this example, contains a lowercase character, `isupper` returns `false`. The `if` statement will cause the message `"Letter is lowercase"` to be displayed.

* These functions actually return an `int` value. The return value is nonzero to indicate `true`, or zero to indicate `false`.

Table 10-1 lists several character-testing functions. Each of these is prototyped in the <cctype> header file, so be sure to include that file when using the functions.

Table 10-1 Character-Testing Functions in <cctype>

Character Function	Description
isalpha	Returns true (a nonzero number) if the argument is a letter of the alphabet. Returns 0 if the argument is not a letter.
isalnum	Returns true (a nonzero number) if the argument is a letter of the alphabet or a digit. Otherwise, it returns 0.
isdigit	Returns true (a nonzero number) if the argument is a digit from 0 through 9. Otherwise, it returns 0.
islower	Returns true (a nonzero number) if the argument is a lowercase letter. Otherwise, it returns 0.
isprint	Returns true (a nonzero number) if the argument is a printable character (including a space). Returns 0 otherwise.
ispunct	Returns true (a nonzero number) if the argument is a printable character other than a digit, letter, or space. Returns 0 otherwise.
isupper	Returns true (a nonzero number) if the argument is an uppercase letter. Otherwise, it returns 0.
isspace	Returns true (a nonzero number) if the argument is a whitespace character. Whitespace characters are any of the following: space ' ' vertical tab '\v' newline '\n' tab '\t' Otherwise, it returns 0.

Program 10-1 uses several of the functions shown in Table 10-1. It asks the user to input a character then displays various messages, depending upon the return value of each function.

Program 10-1

```
1  // This program demonstrates some character-testing functions.
2  #include <iostream>
3  #include <cctype>
4  using namespace std;
5
6  int main()
7  {
8       char input;
9
10      cout << "Enter any character: ";
11      cin.get(input);
12      cout << "The character you entered is: " << input << endl;
```

```
13      if (isalpha(input))
14         cout << "That's an alphabetic character.\n";
15      if (isdigit(input))
16         cout << "That's a numeric digit.\n";
17      if (islower(input))
18         cout << "The letter you entered is lowercase.\n";
19      if (isupper(input))
20         cout << "The letter you entered is uppercase.\n";
21      if (isspace(input))
22         cout << "That's a whitespace character.\n";
23      return 0;
24  }
```

Program Output with Example Input Shown in Bold

```
Enter any character: A Enter
The character you entered is: A
That's an alphabetic character.
The letter you entered is uppercase.
```

Program Output with Different Example Input Shown in Bold

```
Enter any character: 7 Enter
The character you entered is: 7
That's a numeric digit.
```

Program 10-2 shows a more practical application of the character-testing functions. It tests a seven-character customer number to determine whether it is in the proper format.

Program 10-2

```
1   // This program tests a customer number to determine whether
2   // it is in the proper format.
3   #include <iostream>
4   #include <cctype>
5   using namespace std;
6
7   // Function prototype
8   bool testNum(char [], int);
9
10  int main()
11  {
12      const int SIZE = 8;    // Array size
13      char customer[SIZE];   // To hold a customer number
14
15      // Get the customer number.
16      cout << "Enter a customer number in the form ";
17      cout << "LLLNNNN\n";
18      cout << "(LLL = letters and NNNN = numbers): ";
19      cin.getline(customer, SIZE);
20
```

(program continues)

Program 10-2 *(continued)*

```
21      // Determine whether it is valid.
22      if (testNum(customer, SIZE))
23          cout << "That's a valid customer number.\n";
24      else
25      {
26          cout << "That is not the proper format of the ";
27          cout << "customer number.\nHere is an example:\n";
28          cout << "    ABC1234\n";
29      }
30      return 0;
31  }
32
33  //*********************************************************
34  // Definition of function testNum.                        *
35  // This function determines whether the custNum parameter *
36  // holds a valid customer number. The size parameter is   *
37  // the size of the custNum array.                         *
38  //*********************************************************
39
40  bool testNum(char custNum[], int size)
41  {
42      int count; // Loop counter
43
44      // Test the first three characters for alphabetic letters.
45      for (count = 0; count < 3; count++)
46      {
47          if (!isalpha(custNum[count]))
48              return false;
49      }
50
51      // Test the remaining characters for numeric digits.
52      for (count = 3; count < size - 1; count++)
53      {
54          if (!isdigit(custNum[count]))
55              return false;
56      }
57      return true;
58  }
```

Program Output with Example Input Shown in Bold

```
Enter a customer number in the form LLLNNNN
(LLL = letters and NNNN = numbers): RQS4567 Enter
That's a valid customer number.
```

Program Output with Different Example Input Shown in Bold

```
Enter a customer number in the form LLLNNNN
(LLL = letters and NNNN = numbers): AX467T9 Enter
That is not the proper format of the customer number.
Here is an example:
    ABC1234
```

In this program, the customer number is expected to consist of three alphabetic letters followed by four numeric digits. The `testNum` function accepts an array argument and tests the first three characters with the following loop in lines 45 through 49:

```
for (count = 0; count < 3; count++)
{
     if (!isalpha(custNum[count]))
        return false;
}
```

The `isalpha` function returns `true` if its argument is an alphabetic character. The `!` operator is used in the `if` statement to determine whether the tested character is NOT alphabetic. If this is so for any of the first three characters, the function `testNum` returns `false`. Likewise, the next four characters are tested to determine whether they are numeric digits with the following loop in lines 52 through 56:

```
for (count = 3; count < size - 1; count++)
{
     if (!isdigit(custNum[count]))
        return false;
}
```

The `isdigit` function returns `true` if its argument is the character representation of any of the digits 0 through 9. Once again, the `!` operator is used to determine whether the tested character is *not* a digit. If this is so for any of the last four characters, the function `testNum` returns `false`. If the customer number is in the proper format, the function will cycle through both the loops without returning `false`. In that case, the last line in the function is the `return true` statement, which indicates the customer number is valid.

10.2 Character Case Conversion

CONCEPT: The C++ library offers functions for converting a character to uppercase or lowercase.

The C++ library provides two functions, `toupper` and `tolower`, for converting the case of a character. The functions are described in Table 10-2. (These functions are prototyped in the header file `<cctype>`, so be sure to include it.)

Table 10-2 Case-Conversion Functions in `<cctype>`

Function	Description
`toupper`	Returns the uppercase equivalent of its argument.
`tolower`	Returns the lowercase equivalent of its argument.

Each of the functions in Table 10-2 accepts a single character argument. If the argument is a lowercase letter, the `toupper` function returns its uppercase equivalent. For example, the following statement will display the character `A` on the screen:

```
cout << toupper('a');
```

If the argument is already an uppercase letter, toupper returns it unchanged. The following statement causes the character Z to be displayed:

```
cout << toupper('Z');
```

Any nonletter argument passed to toupper is returned as it is. Each of the following statements displays toupper's argument without any change:

```
cout << toupper('*');    // Displays *
cout << toupper ('&');   // Displays &
cout << toupper('%');    // Displays %
```

toupper and tolower don't actually cause the character argument to change; they simply return the uppercase or lowercase equivalent of the argument. For example, in the following program segment, the variable letter is set to the value 'A'. The tolower function returns the character 'a', but letter still contains 'A'.

```
char letter = 'A';
cout << tolower(letter) << endl;
cout << letter << endl;
```

These statements will cause the following to be displayed:

```
a
A
```

Program 10-3 demonstrates the toupper function in an input validation loop.

Program 10-3

```
1  // This program calculates the area of a circle. It asks the user
2  // if he or she wishes to continue. A loop that demonstrates the
3  // toupper function repeats until the user enters 'y', 'Y',
4  // 'n', or 'N'.
5  #include <iostream>
6  #include <cctype>
7  #include <iomanip>
8  using namespace std;
9
10 int main()
11 {
12     const double PI = 3.14159;  // Constant for pi
13     double radius;              // The circle's radius
14     char goAgain;               // To hold Y or N
15
16     cout << "This program calculates the area of a circle.\n";
17     cout << fixed << setprecision(2);
18
19     do
20     {
21         // Get the radius and display the area.
22         cout << "Enter the circle's radius: ";
23         cin >> radius;
24         cout << "The area is " << (PI * radius * radius);
25         cout << endl;
```

```
26
27          // Does the user want to do this again?
28          cout << "Calculate another? (Y or N) ";
29          cin >> goAgain;
30
31          // Validate the input.
32          while (toupper(goAgain) != 'Y' && toupper(goAgain) != 'N')
33          {
34               cout << "Please enter Y or N: ";
35               cin >> goAgain;
36          }
37
38      } while (toupper(goAgain) == 'Y');
39      return 0;
40  }
```

Program Output with Example Input Shown in Bold

```
This program calculates the area of a circle.
Enter the circle's radius: 10 Enter
The area is 314.16
Calculate another? (Y or N) b Enter
Please enter Y or N: y Enter
Enter the circle's radius: 1 Enter
The area is 3.14
Calculate another? (Y or N) n Enter
```

In lines 28 and 29, the user is prompted to enter either Y or N to indicate whether he or she wants to calculate another area. We don't want the program to be so picky that it accepts only uppercase Y or uppercase N. Lowercase y or lowercase n is also acceptable. The input validation loop must be written to reject anything except `'Y'`, `'y'`, `'N'`, or `'n'`. One way to do this would be to test the `goAgain` variable in four relational expressions, as shown here:

```
while (goAgain != 'Y' && goAgain != 'y' &&
       goAgain != 'N' && goAgain != 'N')
```

Although there is nothing wrong with this code, we could use the `toupper` function to get the uppercase equivalent of `goAgain` and make only two comparisons. This is the approach taken in line 32:

```
while (toupper(goAgain) != 'Y' && toupper(goAgain) != 'N')
```

Another approach would have been to use the `tolower` function to get the lowercase equivalent of `goAgain`. Here is an example:

```
while (tolower(goAgain) != 'y' && tolower(goAgain) != 'n')
```

Either approach will yield the same results.



Checkpoint

10.1 Write a short description of each of the following functions:

```
isalpha
isalnum
isdigit
islower
isprint
ispunct
isupper
isspace
toupper
tolower
```

10.2 Write a statement that will convert the contents of the `char` variable `big` to lowercase. The converted value should be assigned to the variable `little`.

10.3 Write an `if` statement that will display the word "digit" if the variable `ch` contains a numeric digit. Otherwise, it should display "Not a digit."

10.4 What is the output of the following statement?

```
cout << toupper(tolower('A'));
```

10.5 Write a loop that asks the user "Do you want to repeat the program or quit? (R/Q)". The loop should repeat until the user has entered an R or a Q (either uppercase or lowercase).

10.3 C-Strings

CONCEPT: In C++, a C-string is a sequence of characters stored in consecutive memory locations, terminated by a null character.

String is a generic term that describes any consecutive sequence of characters. A word, a sentence, a person's name, and the title of a song are all strings. In the C++ language, there are two primary ways that strings are stored in memory: as `string` objects or as C-strings. You have already been introduced to the `string` class, and by now, you have written several programs that use `string` objects. In this section, we will use C-strings, which are an alternative method for storing and working with strings.

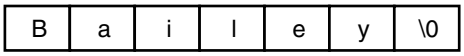
A *C-string* is a string whose characters are stored in consecutive memory locations and are followed by a null character, or null terminator. Recall from Chapter 2 that a null character or null terminator is a byte holding the ASCII code 0. Strings that are stored this way are called C-strings because this is the way strings are handled in the C programming language.

In C++, all string literals are stored in memory as C-strings. Recall that a string literal (or string constant) is the literal representation of a string in a program. In C++, string literals are enclosed in double quotation marks, such as:

```
"Bailey"
```

Figure 10-1 illustrates how the string literal `"Bailey"` is stored in memory, as a C-string.

Figure 10-1 A C-string stored in memory





NOTE: Remember that `\0` ("slash zero") is the escape sequence representing the null terminator. It stands for the ASCII code 0.

The purpose of the null terminator is to mark the end of the C-string. Without it, there would be no way for a program to know the length of a C-string.

More about String Literals

A string literal or string constant is enclosed in a set of double quotation marks (`" "`). For example, here are five string literals:

```
"Have a nice day."
"What is your name?"
"John Smith"
"Please enter your age:"
"Part Number 45Q1789"
```

All of a program's string literals are stored in memory as C-strings, with the null terminator automatically appended. For example, look at Program 10-4.

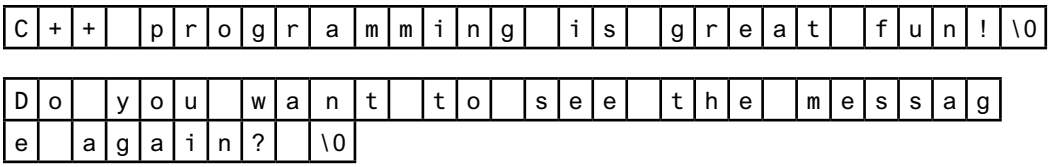
Program 10-4

```
1  // This program contains string literals.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7       char again;
8
9       do
10      {
11            cout << "C++ programming is great fun!" << endl;
12            cout << "Do you want to see the message again? ";
13            cin >> again;
14      } while (again == 'Y' || again == 'y');
15      return 0;
16 }
```

This program contains two string literals:

```
"C++ programming is great fun!"
"Do you want to see the message again? "
```

The first string occupies 30 bytes of memory (including the null terminator), and the second string occupies 39 bytes. They appear in memory in the following forms:



It's important to realize that a string literal has its own storage location, just like a variable or an array. When a string literal appears in a statement, it's actually its memory address that C++ uses. Look at the following example:

```
cout << "Do you want to see the message again? ";
```

In this statement, the memory address of the string literal "Do you want to see the message again?" is passed to the `cout` object. `cout` displays the consecutive characters found at this address. It stops displaying the characters when a null terminator is encountered.

C-Strings Stored in Arrays

The C programming language does not provide a `string` class like that which C++ provides. In the C language, all strings are treated as C-strings. When a C programmer wants to store a string in memory, he or she has to create a `char` array that is large enough to hold the string, plus one extra element for the null character.

You might be wondering why this should matter to anyone learning C++. You need to know about C-strings for the following reasons:

- The `string` class has not always existed in the C++ language. Several years ago, C++ stored strings as C-strings. As a professional programmer, you might encounter older C++ code (known as *legacy code*) that uses C-strings.
- Some of the C++ library functions work only with C-strings.
- In the workplace, it is not unusual for C++ programmers to work with specialized libraries that are written in C. Any strings with which C libraries work will be C-strings.

As previously mentioned, if you want to store a C-string in memory, you have to define a `char` array that is large enough to hold the string, plus one extra element for the null character. Here is an example:

```
const int SIZE = 21;
char name[SIZE];
```

This code defines a `char` array that has 21 elements, so it is big enough to hold a C-string that is no more that 20 characters long.

You can initialize a `char` array with a string literal, as shown here:

```
const int SIZE = 21;
char name[SIZE] = "Jasmine";
```

After this code executes, the `name` array will be created with 21 elements. The first eight elements will be initialized with the characters `'J'`, `'a'`, `'s'`, `'m'`, `'i'`, `'n'`, `'e'`, and `'\0'`. The null character is automatically added as the last character. You can implicitly size a `char` array by initializing it with a string literal, as shown here:

```
char name[] = "Jasmine";
```

After this code executes, the name array will be created with eight elements, initialized with the characters 'J','a', 's', 'm', 'i', 'n', 'e', and '\0'.

C-string input can be performed by the cin object. For example, the following code allows the user to enter a string (with no whitespace characters) into the name array:

```
const int SIZE = 21;
char name[SIZE];
cin >> name;
```

Recall from Chapter 7 that an array name with no brackets and no subscript is converted into the beginning address of the array. In the previous statement, name indicates the address in memory where the string is to be stored. Of course, cin has no way of knowing that name has 21 elements. If the user enters a string of 30 characters, cin will write past the end of the array. This can be prevented by using cin's getline member function. Assume the following array has been defined in a program:

```
const int SIZE = 80;
char line[SIZE];
```

The following statement uses cin's getline member function to get a line of input (including whitespace characters) and store it in the line array:

```
cin.getline(line, SIZE);
```

The first argument tells getline where to store the string input. This statement indicates the starting address of the line array as the storage location for the string. The second argument indicates the maximum length of the string, including the null terminator. In this example, the SIZE constant is equal to 80, so cin will read 79 characters, or until the user presses the Enter key, whichever comes first. cin will automatically append the null terminator to the end of the string.

Once a string is stored in an array, it can be processed using standard subscript notation. For example, Program 10-5 displays a string stored in an array. It uses a loop to display each character in the array until the null terminator is encountered.

Program 10-5

```
1  // This program displays a string stored in a char array.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      const int SIZE = 80;  // Array size
8      char line[SIZE];      // To hold a line of input
9      int count = 0;        // Loop counter variable
10
11     // Get a line of input.
12     cout << "Enter a sentence of no more than "
13          << (SIZE - 1) << " characters:\n";
14     cin.getline(line, SIZE);
15
```

(program continues)

Program 10-5 *(continued)*

```
16      // Display the input one character at a time.
17      cout << "The sentence you entered is:\n";
18      while (line[count] != '\0')
19      {
20           cout << line[count];
21           count++;
22      }
23      return 0;
24  }
```

Program Output with Example Input Shown in Bold

```
Enter a sentence of no more than 79 characters:
C++ is challenging but fun! Enter
The sentence you entered is:
C++ is challenging but fun!
```

10.4 Library Functions for Working with C-Strings

CONCEPT: The C++ library has numerous functions for handling C-strings. These functions perform various tests and manipulations, and require the `<cstring>` header file be included.

The `strlen` Function

Because C-strings are stored in arrays, working with them is quite different from working with `string` objects. Fortunately, the C++ library provides many functions for manipulating and testing C-strings. These functions all require the `<cstring>` header file to be included, as shown here:

```
#include <cstring>
```

For instance, the following code segment uses the `strlen` function to determine the length of the string stored in the `name` array:

```
char name[] = "Thomas Edison";
int length;
length = strlen(name);
```

The `strlen` function accepts a pointer to a C-string as its argument. It returns the length of the string, which is the number of characters up to, but not including, the null terminator. As a result, the variable `length` will have the number 13 stored in it. The length of a string isn't to be confused with the size of the array holding it. Remember, the only information being passed to `strlen` is the beginning address of a C-string. It doesn't know where the array ends, so it looks for the null terminator to indicate the end of the string.

When using a C-string-handling function, you must pass one or more C-strings as arguments. This means passing the address of the C-string, which may be accomplished by using any of the following as arguments:

- The name of the array holding the C-string
- A pointer variable that holds the address of the C-string
- A literal string

Anytime a literal string is used as an argument to a function, the address of the literal string is passed. Here is an example of the `strlen` function being used with such an argument:

```
length = strlen("Thomas Edison");
```

The `strcat` Function

The `strcat` function accepts two pointers to C-strings as its arguments. The function *concatenates*, or appends one string to another. The following code shows an example of its use:

```
const int SIZE = 13;
char string1[SIZE] = "Hello ";
char string2[] = "World!";

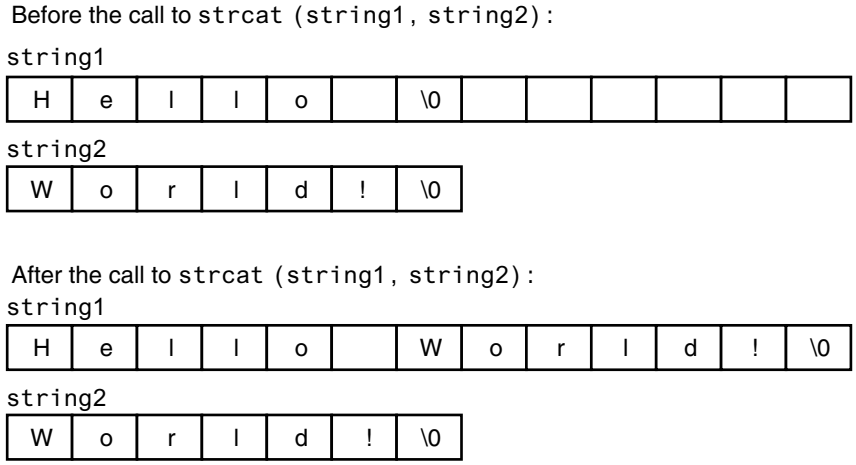
cout << string1 << endl;
cout << string2 << endl;
strcat(string1, string2);
cout << string1 << endl;
```

These statements will cause the following output:

```
Hello
World!
Hello World!
```

The `strcat` function copies the contents of `string2` to the end of `string1`. In this example, `string1` contains the string "Hello " before the call to `strcat`. After the call, it contains the string "Hello World!". Figure 10-2 shows the contents of both arrays before and after the function call.

Figure 10-2 Using `strcat`



Notice the last character in string1 (before the null terminator) is a space. The strcat function doesn't insert a space, so it's the programmer's responsibility to make sure one is already there, if needed. It's also the programmer's responsibility to make sure the array holding string1 is large enough to hold string1 plus string2 plus a null terminator.

Here is a program segment that uses the sizeof operator to test an array's size before strcat is called:

```
if (sizeof(string1) >= (strlen(string1) + strlen(string2) + 1))
    strcat(string1, string2);
else
    cout << "String1 is not large enough for both strings.\n";
```



WARNING! If the array holding the first string isn't large enough to hold both strings, strcat will overflow the boundaries of the array.

The strcpy Function

Recall from Chapter 7 that one array cannot be assigned to another with the = operator. Each individual element must be assigned, usually inside a loop. The strcpy function can be used to copy one string to another. Here is an example of its use:

```
const int SIZE = 13;
char name[SIZE];
strcpy(name, "Albert Einstein");
```

The strcpy function's two arguments are C-string addresses. The contents of the second argument are copied to the memory location specified by the first argument, including the null terminator. (The first argument usually references an array.) In this example, the strcpy function will copy the string "Albert Einstein" to the name array.

If anything is already stored in the location referenced by the first argument, it is overwritten, as shown in the following program segment:

```
const int SIZE = 10;
char string1[SIZE] = "Hello", string2[SIZE] = "World!";
cout << string1 << endl;
cout << string2 << endl;
strcpy(string1, string2);
cout << string1 << endl;
cout << string2 << endl;
```

Here is the output:

```
Hello
World!
World!
World!
```



WARNING! Being true to C++'s nature, strcpy performs no bounds checking. The array specified by the first argument will be overflowed if it isn't large enough to hold the string specified by the second argument.

The strncat and strncpy Functions

Because the strcat and strcpy functions can potentially overwrite the bounds of an array, they make it possible to write unsafe code. As an alternative, you should use strncat and strncpy whenever possible.

The strncat functions works like strcat, except it takes a third argument specifying the maximum number of characters from the second string to append to the first. Here is an example call to strncat:

```
strncat(string1, string2, 10);
```

When this statement executes, strncat will append no more than 10 characters from string2 to string1. The following code shows an example of calculating the maximum number of characters that can be appended to an array:

```
1  int maxChars;
2  const int SIZE_1 = 17;
3  const int SIZE_2 = 18;
4
5  char string1[SIZE_1] = "Welcome ";
6  char string2[SIZE_2] = "to North Carolina";
7
8  cout << string1 << endl;
9  cout << string2 << endl;
10 maxChars = sizeof(string1) - (strlen(string1) + 1);
11 strncat(string1, string2, maxChars);
12 cout << string1 << endl;
```

The statement in line 10 calculates the number of empty elements in string1. It does this by subtracting the length of the string stored in the array plus 1 for the null terminator. This code will cause the following output:

```
Welcome
to North Carolina
Welcome to North
```

The strncpy function allows you to copy a specified number of characters from a string to a destination. Calling strncpy is similar to calling strcpy, except you pass a third argument specifying the maximum number of characters from the second string to copy to the first. Here is an example call to strncpy:

```
strncpy(string1, string2, 5);
```

When this statement executes, strncpy will copy no more than five characters from string2 to string1. However, if the specified number of characters is less than or equal to the length of string2, a null terminator is not appended to string1. If the specified number of characters is greater than the length of string2, then string1 is padded with null terminators, up to the specified number of characters. The following code shows an example using the strncpy function:

```
1  int maxChars;
2  const int SIZE = 11;
3
4  char string1[SIZE];
5  char string2[] = "I love C++ programming!";
6
```

```
 7  maxChars = sizeof(string1) - 1;
 8  strncpy(string1, string2, maxChars);
 9  // Put the null terminator at the end.
10  string1[maxChars] = '\0';
11  cout << string1 << endl;
```

Notice a statement was written in line 10 to put the null terminator at the end of `string1`. This is because `maxChars` was less than the length of `string2`, and `strncpy` did not automatically place a null terminator there.

The `strstr` Function

The `strstr` function searches for a string inside of a string. For instance, it could be used to search for the string "seven" inside the larger string "Four score and seven years ago." The function's first argument is the string to be searched, and the second argument is the string for which to look. If the function finds the second string inside the first, it returns the address of the occurrence of the second string within the first string. Otherwise, it returns `nullptr` (the address 0). Here is an example:

```
char arr[] = "Four score and seven years ago";
char *strPtr = nullptr;
cout << arr << endl;
strPtr = strstr(arr, "seven"); // search for "seven"
cout << strPtr << endl;
```

In this code, `strstr` will locate the string "seven" inside the string "Four score and seven years ago." It will return the address of the first character in "seven" which will be stored in the pointer variable `strPtr`. If run as part of a complete program, this segment will display the following:

```
Four score and seven years ago
seven years ago
```



NOTE: The `nullptr` key word was introduced in C++ 11. If you are using a previous version of the C++ language, use the constant `NULL` instead.

The `strstr` function can be useful in any program that must locate data inside one or more strings. Program 10-6, for example, stores a list of product numbers and descriptions in an array of C-strings. It allows the user to look up a product description by entering all or part of its product number.

Program 10-6

```
 1  // This program uses the strstr function to search an array.
 2  #include <iostream>
 3  #include <cstring>    // For strstr
 4  using namespace std;
 5
 6  int main()
 7  {
 8      // Constants for array lengths
 9      const int NUM_PRODS = 5;    // Number of products
10      const int LENGTH = 27;      // String length
```

```
11
12      // Array of products
13      char products[NUM_PRODS][LENGTH] =
14                      { "TV327 31-inch Television",
15                        "CD257 CD Player",
16                        "TA677 Answering Machine",
17                        "CS109 Car Stereo",
18                        "PC955 Personal Computer" };
19
20      char lookUp[LENGTH];      // To hold user's input
21      char *strPtr = nullptr;   // To point to the found product
22      int index;                // Loop counter
23
24      // Prompt the user for a product number.
25      cout << "\tProduct Database\n\n";
26      cout << "Enter a product number to search for: ";
27      cin.getline(lookUp, LENGTH);
28
29      // Search the array for a matching substring
30      for (index = 0; index < NUM_PRODS; index++)
31      {
32          strPtr = strstr(products[index], lookUp);
33          if (strPtr != nullptr)
34              break;
35      }
36
37      // If a matching substring was found, display the product info.
38      if (strPtr != nullptr)
39          cout << products[index] << endl;
40      else
41          cout << "No matching product was found.\n";
42
43      return 0;
44  }
```

Program Output with Example Input Shown in Bold

```
      Product Database

Enter a product to search for: CS [Enter]
CS109 Car Stereo
```

Program Output with Different Example Input Shown in Bold

```
      Product Database

Enter a product to search for: AB [Enter]
No matching product was found.
```

Table 10-3 summarizes the string-handling functions discussed here, as well as the `strcmp` function that was discussed in Chapter 4. (All the functions listed require the `<cstring>` header file.)

In Program 10-6, the `for` loop in lines 30 through 35 steps through each C-string in the array calling the following statement:

```
strPtr = strstr(prods[index], lookUp);
```

The strstr function searches the string referenced by prods[index] for the name entered by the user, which is stored in lookUp. If lookUp is found inside prods[index], the function returns its address. In that case, the following if statement causes the for loop to terminate:

```
if (strPtr != nullptr)
    break;
```

Outside the loop, the following if else statement in lines 38 through 41 determines whether the string entered by the user was found in the array. If not, it informs the user that no matching product was found. Otherwise, the product number and description are displayed.

```
if (strPtr == nullptr)
    cout << "No matching product was found.\n";
else
    cout << prods[index] << endl;
```

The strcmp Function

Because C-strings are stored in char arrays, you cannot use the relational operators to compare two C-strings. To compare C-strings, you should use the library function strcmp. This function takes two C-strings as arguments and returns an integer that indicates how the two strings compare to each other. Here is the function's prototype:

```
int strcmp(char *string1, char *string2);
```

The function takes two C-strings as parameters (actually, pointers to C-strings) and returns an integer result. The value of the result is set accordingly:

- The result is *zero* if the two strings are *equal* on a character-by-character basis.
- The result is *negative* if string1 comes *before* string2 in alphabetical order.
- The result is *positive* if string1 comes *after* string2 in alphabetical order.

Here is an example of the use of strcmp to determine if two strings are equal:

```
if (strcmp(string1, string2) == 0)
    cout << "The strings are equal.\n";
else
    cout << "The strings are not equal.\n";
```

Program 10-7 shows a complete example.

Program 10-7

```
1  // This program tests two C-strings for equality
2  // using the strcmp function.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main()
8  {
9      // Two arrays for two strings.
10     const int LENGTH = 40;
11     char firstString[LENGTH], secondString[LENGTH];
12
```

```
13      // Read two strings.
14      cout << "Enter a string: ";
15      cin.getline(firstString, LENGTH);
16      cout << "Enter another string: ";
17      cin.getline(secondString, LENGTH);
18
19      // Compare the strings for equality with strcmp.
20      if (strcmp(firstString, secondString) == 0)
21          cout << "You entered the same string twice.\n";
22      else
23          cout << "The strings are not the same.\n";
24
25      return 0;
26  }
```

Program Output with Example Input Shown in Bold

```
Enter a string: Alfonso [Enter]
Enter another string: Alfonso [Enter]
You entered the same string twice.
```

The `strcmp` function is case sensitive when it compares strings. If the user enters "Dog" and "dog" in Program 10-7, it will report they are not the same. Most compilers provide nonstandard versions of `strcmp` that perform case-insensitive comparisons. For instance, some compilers provide a function named `stricmp` that works identically to `strcmp`, except the case of the characters is ignored.

Program 10-8 is a more practical example of how `strcmp` can be used. It asks the user to enter the part number of the MP3 player they wish to purchase. The part number contains digits, letters, and a hyphen, so it must be stored as a string. Once the user enters the part number, the program displays the price of the stereo.

Program 10-8

```
1   // This program uses strcmp to compare the string entered
2   // by the user with the valid MP3 player part numbers.
3   #include <iostream>
4   #include <cstring>
5   #include <iomanip>
6   using namespace std;
7
8   int main()
9   {
10      // Price of parts.
11      const double A_PRICE = 99.0,
12                   B_PRICE = 199.0;
13
14      // Character array for part number.
15      const int PART_LENGTH = 9;
16      char partNum[PART_LENGTH];
17
```

(program continues)

Program 10-8 *(continued)*

```
18       // Instruct the user to enter a part number.
19       cout << "The MP3 player part numbers are:\n"
20            << "\t16 Gigabyte, part number S147-29A\n"
21            << "\t32 Gigabyte, part number S147-29B\n"
22            << "Enter the part number of the MP3 player you\n"
23            << "wish to purchase: ";
24
25       // Read a part number of at most 8 characters.
26       cin >> partNum;
27
28       // Determine what user entered using strcmp
29       // and print its price.
30       cout << showpoint << fixed << setprecision(2);
31       if (strcmp(partNum, "S147-29A") == 0)
32           cout << "The price is $" << A_PRICE << endl;
33       else if (strcmp(partNum, "S147-29B") == 0)
34           cout << "The price is $" << B_PRICE << endl;
35       else
36           cout << partNum << " is not a valid part number.\n";
37       return 0;
38   }
```

Program Output with Example Input Shown in Bold

```
The MP3 player part numbers are:
    16 Gigabyte, part number S147-29A
    32 Gigabyte, part number S147-29B
Enter the part number of the stereo you
wish to purchase: S147-29B [Enter]
The price is $199.00
```

Using ! with strcmp

Some programmers prefer to use the logical NOT operator with `strcmp` when testing strings for equality. Because 0 is considered logically false, the `!` operator converts that value to true. The expression `!strcmp(string1, string2)` returns `true` when both strings are the same, and false when they are different. The two following statements perform the same operation:

```
if (strcmp(firstString, secondString) == 0)
if (!strcmp(firstString, secondString))
```

Sorting Strings

Programs are frequently written to print alphabetically sorted lists of items. For example, consider a department store computer system that keeps customers' names and addresses in a file. The names do not appear in the file alphabetically, but in the order the operator entered them. If a list were to be printed in this order, it would be very difficult to locate any specific name. The list would have to be sorted before it was printed.

Because the value returned by `strcmp` is based on the relative alphabetic order of the two strings being compared, it can be used in programs that sort strings. Program 10-9 asks the user to enter two names, which are then printed in alphabetic order.

Program 10-9

```
1  // This program uses the return value of strcmp to
2  // alphabetically sort two strings entered by the user.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main()
8  {
9        // Two arrays to hold two strings.
10       const int NAME_LENGTH = 30;
11       char name1[NAME_LENGTH], name2[NAME_LENGTH];
12
13       // Read two strings.
14       cout << "Enter a name (last name first): ";
15       cin.getline(name1, NAME_LENGTH);
16       cout << "Enter another name: ";
17       cin.getline(name2, NAME_LENGTH);
18
19       // Print the two strings in alphabetical order.
20       cout << "Here are the names sorted alphabetically:\n";
21       if (strcmp(name1, name2) < 0)
22           cout << name1 << endl << name2 << endl;
23       else if (strcmp(name1, name2) > 0)
24           cout << name2 << endl << name1 << endl;
25       else
26           cout << "You entered the same name twice!\n";
27
28       return 0;
29 }
```

Program Output with Example Input Shown in Bold

```
Enter a name (last name first): Smith, Richard [Enter]
Enter another name: Jones, John [Enter]
Here are the names sorted alphabetically:
Jones, John
Smith, Richard
```

Table 10-3 provides a summary of the C-string-handling functions we have discussed. All of the functions listed require the `<cstring>` header file.

Table 10-3 Some of the C-String Functions in `<cstring>`

Function	Description
`strlen`	Accepts a C-string or a pointer to a C-string as an argument. Returns the length of the C-string (not including the null terminator.) *Example Usage:* `len = strlen(name);`
`strcat`	Accepts two C-strings or pointers to two C-strings as arguments. The function appends the contents of the second string to the first C-string. (The first string is altered, the second string is left unchanged.) *Example Usage:* `strcat(string1, string2);`

(table continues)

Table 10-3 *(continued)*

Function	Description
`strcpy`	Accepts two C-strings or pointers to two C-strings as arguments. The function copies the second C-string to the first C-string. The second C-string is left unchanged. *Example Usage:* `strcpy(string1, string2);`
`strncat`	Accepts two C-strings or pointers to two C-strings, and an integer argument. The third argument, an integer, indicates the maximum number of characters to copy from the second C-string to the first C-string. *Example Usage:* `strncat(string1, string2, n);`
`strncpy`	Accepts two C-strings or pointers to two C-strings, and an integer argument. The third argument, an integer, indicates the maximum number of characters to copy from the second C-string to the first C-string. If `n` is less than the length of `string2`, the null terminator is not automatically appended to `string1`. If `n` is greater than the length of `string2`, `string1` is padded with '\0' characters. *Example Usage:* `strncpy(string1, string2, n);`
`strcmp`	Accepts two C-strings or pointers to two C-strings arguments. If `string1` and `string2` are the same, this function returns 0. If `string2` is alphabetically greater than `string1`, it returns a negative number. If `string2` is alphabetically less than `string1`, it returns a positive number. *Example Usage:* `if (strcmp(string1, string2))`
`strstr`	Accepts two C-strings or pointers to two C-strings as arguments. Searches for the first occurrence of `string2` in `string1`. If an occurrence of `string2` is found, the function returns a pointer to it. Otherwise, it returns `nullptr` (address 0). *Example Usage:* `cout << strstr(string1, string2);`



Checkpoint

10.6 Write a short description of each of the following functions:

```
strlen
strcat
strcpy
strncat
strncpy
strcmp
strstr
```

10.7 What will the following program segment display?

```
char dog[] = "Fido";
cout << strlen(dog) << endl;
```

10.8 What will the following program segment display?

```
char string1[16] = "Have a ";
char string2[9] = "nice day";
strcat(string1, string2);
cout << string1 << endl;
cout << string2 << endl;
```

10.9 Write a statement that will copy the string "Beethoven" to the array `composer`.

10.10 When complete, the following program skeleton will search for the string "Windy" in the array `place`. If `place` contains "Windy" the program will display the message "Windy found." Otherwise, it will display "Windy not found."

```
#include <iostream>
// include any other necessary header files
using namespace std;

int main()
{
    char place[] = "The Windy City";
    // Complete the program. It should search the array place
    // for the string "Windy" and display the message "Windy
    // found" if it finds the string. Otherwise, it should
    // display the message "Windy not found."
    return 0;
}
```

10.5 String/Numeric Conversion Functions

CONCEPT: The C++ library provides functions for converting C-strings and `string` objects to numeric data types and vice versa.

There is a great difference between a number that is stored as a string, and one stored as a numeric value. The string "`26792`" isn't actually a number, but a series of ASCII codes representing the individual digits of the number. It uses 6 bytes of memory (including the null terminator). Because it isn't an actual number, it's not possible to perform mathematical operations with it, unless it is first converted to a numeric value.

Several functions exist in the C++ library for converting C-string representations of numbers into numeric values. Table 10-4 shows some of these. Note all of these functions require the `<cstdlib>` header file.

Table 10-4 C-String/Numeric Conversion Functions in `<cstdlib>`

Function	Description
`atoi`	Accepts a C-string as an argument. The function converts the C-string to an integer and returns that value. *Example Usage:* `int num = atoi("4569");`
`atol`	Accepts a C-string as an argument. The function converts the C-string to a `long` integer and returns that value. *Example Usage:* `long lnum = atol("500000");`
`atof`	Accepts a C-string as an argument. The function converts the C-string to a `double` and returns that value. *Example Usage:* `double fnum = atof("3.14159");`



NOTE: If a C-string that cannot be converted to a numeric value is passed to any of the functions in Table 10-4, the function's behavior is undefined by C++. Many compilers, however, will perform the conversion process until an invalid character is encountered. For example, `atoi("123x5")` might return the integer 123. It is possible that these functions will return 0 if they cannot successfully convert their argument.

The string to Number Functions



C++ 11 introduced several new functions that convert `string` objects to numeric values. Table 10-5 lists the functions. To use any of the functions in Table 10-5, you must `#include` the `<string>` header file.

Table 10-5 `string` to Number Functions

Function	Description
`stoi(string str)`	Accepts a `string` argument and returns that argument's value converted to an `int`.
`stol(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long`.
`stoul(string str)`	Accepts a `string` argument and returns that argument's value converted to an `unsigned long`.
`stoll(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long long`.
`stoull(string str)`	Accepts a `string` argument and returns that argument's value converted to an `unsigned long long`.
`stof(string str)`	Accepts a `string` argument and returns that argument's value converted to a `float`.
`stod(string str)`	Accepts a `string` argument and returns that argument's value converted to a `double`.
`stold(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long double`.



NOTE: If a `string` that cannot be converted to a numeric value is passed to any of the functions in Table 10-5, the function will throw an `invalid_argument` exception. If the `string` argument converts to a value that is out of range for the target data type, the function will throw an `out_of_range` exception.

The functions shown in Table 10-5 can accept either a `string` object or a C-string as an argument. For example, each of the following code snippets use the `stoi` function to convert the string "99" to an `int`:

```
// Convert a string object to an int.
string str = "99";
int i = stoi(str);

// Convert a string literal to an int.
int i = stoi("99");
```

```
// Convert a C-String in a char array to an int.
char cstr[] = "99";
int i = stoi(cstr);
```

The to_string Function



C++ 11 introduces a function named to_string that converts a numeric value to a string object. There are nine overloaded versions of the to_string function listed in Table 10-6. Note the to_string function requires the <string> header file to be included.

Each version of the function accepts an argument of a numeric data type and returns the value of that argument converted to a string object. Here is an example:

```
int number = 99;
string output = to_string(number);
```

Table 10-6 Overloaded Versions of the to_string Function

Function	Description
to_string(int *value*);	Accepts an int argument and returns that argument converted to a string object.
to_string(long *value*);	Accepts a long argument and returns that argument converted to a string object.
to_string(long long *value*);	Accepts a long long argument and returns that argument converted to a string object.
to_string(unsigned *value*);	Accepts an unsigned argument and returns that argument converted to a string object.
to_string(unsigned long *value*);	Accepts an unsigned long argument and returns that argument converted to a string object.
to_string(unsigned long long *value*);	Accepts an unsigned long long argument and returns that argument converted to a string object.
to_string(float *value*);	Accepts a float argument and returns that argument converted to a string object.
to_string(double *value*);	Accepts a double argument and returns that argument converted to a string object.
to_string(long double *value*);	Accepts a long double argument and returns that argument converted to a string object.

The first statement initializes the number variable (an int) to the value 99. In the second statement, the number variable is passed as an argument to the to_string function. The to_string function returns the value "99", which is assigned to the output variable.

Here is another example:

```
double number = 3.14159;
cout << to_string(number) << endl;
```

The first statement initializes the number variable (a double) to the value 3.14159. In the second statement, the number variable is passed as an argument to the to_string function, and the value "3.14159" is returned from the function and displayed on the screen.

Now let's look at Program 10-10, which uses one of the C++ 11 string-to-number conversion functions, stoi. It allows the user to enter a series of values, or the letter Q or q to quit. The average of the numbers is then calculated and displayed.

Program 10-10

```
// This program demonstrates the tolower and stoi functions.
#include <iostream>
#include <cctype>         // For tolower
#include <string>
using namespace std;

int main()
{
   string input;          // To hold user input
   int total = 0;         // Accumulator
   int count = 0;         // Loop counter
   double average;        // To hold the average of numbers

   // Get the first number.
   cout << "This program will average a series of numbers.\n";
   cout << "Enter the first number or Q to quit: ";
   getline(cin, input);

   // Process the number and subsequent numbers.
   while (tolower(input[0]) != 'q')
   {
      total += stoi(input);     // Keep a running total
      count++;                  // Count the numbers entered
      // Get the next number.
      cout << "Enter the next number or Q to quit: ";
      getline(cin, input);
   }

   // If any numbers were entered, display their average.
   if (count != 0)
   {
      average = static_cast<double>(total) / count;
      cout << "Average: " << average << endl;
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
This program will average a series of numbers.
Enter the first number or Q to quit: 1 Enter
Enter the next number or Q to quit: 2 Enter
Enter the next number or Q to quit: 3 Enter
Enter the next number or Q to quit: 4 Enter
Enter the next number or Q to quit: 5 Enter
Enter the next number or Q to quit: q Enter
Average: 3
```

In line 20, the following `while` statement uses the `tolower` function to determine whether the first character entered by the user is "`q`" or "`Q`".

```
while (tolower(input[0]) != 'q')
```

If the user hasn't entered 'Q' or 'q', the loop performs an iteration. The following statement, in line 22, uses `stoi` to convert `input` to an integer, and adds its value to `total`:

```
total += stoi(input);    // Keep a running total
```

The counter is updated in line 23, then the user is asked for the next number. When all the numbers are entered, the user terminates the loop by entering '`Q`' or '`q`'. If one or more numbers are entered, their average is displayed.

The string-to-numeric conversion functions can also help with a common input problem. Recall from Chapter 3 that using `cin >>` then calling `cin.get` causes problems because the `>>` operator leaves the newline character in the keyboard buffer. When the `cin.get` function executes, the first character it sees in the keyboard buffer is the newline character, so it reads no further.

The same problem exists when a program uses `cin >>` then calls `getline` to read a line of input. For example, look at the following code. (Assume `idNumber` is an `int` and `name` is a `string` object.)

```
1  // Get the user's ID number.
2  cout << "What is your ID number? ";
3  cin >> idNumber;
4
5  // Get the user's name.
6  cout << "What is your name? ";
7  getline(cin, name);
```

Let's say the user enters 25 and presses Enter when the `cin >>` statement in line 3 executes. The value 25 will be stored in `idNumber`, and the newline character will be left in the keyboard buffer. When the `getline` function is called in line 7, the first character it sees in the keyboard buffer is the newline character, so it reads no further. It will appear that the statement in line 7 was skipped.

One work-around that we have used in this book is to call `cin.ignore` to skip over the newline character just before calling `getline`. Another approach is to use `getline` to read *all* of a program's input, including numbers. When numeric input is needed, it is read into a `string` object as a string then converted to the appropriate numeric data type. Because you aren't mixing `cin >>` with `getline`, the problem of the remaining newline character doesn't exist. Program 10-11 shows an example.

Program 10-11

```
1  // This program demonstrates how the getline function can
2  // be used for all of a program's input.
3  #include <iostream>
4  #include <string>
5  #include <iomanip>
6  using namespace std;
7
```

(program continues)

Program 10-11 *(continued)*

```
 8  int main()
 9  {
10     string input;        // To hold a line of input
11     string name;         // To hold a name
12     int idNumber;        // To hold an ID number.
13     int age;             // To hold an age
14     double income;       // To hold income
15
16     // Get the user's ID number.
17     cout << "What is your ID number? ";
18     getline(cin, input);             // Read as a string
19     idNumber = stoi(input);          // Convert to int
20
21     // Get the user's name. No conversion necessary.
22     cout << "What is your name? ";
23     getline(cin, name);
24
25     // Get the user's age.
26     cout << "How old are you? ";
27     getline(cin, input);             // Read as a string
28     age = stoi(input);               // Convert to int
29
30     // Get the user's income.
31     cout << "What is your annual income? ";
32     getline(cin, input);             // Read as a string
33     income = stod(input);            // Convert to double
34
35     // Show the resulting data.
36     cout << setprecision(2) << fixed << showpoint;
37     cout << "Your name is " << name
38          <<", you are " << age
39          << " years old,\nand you make $"
40          << income << " per year.\n";
41
42     return 0;
43  }
```

Program Output with Example Input Shown in Bold

```
What is your ID number? 1234 [Enter]
What is your name? Janice Smith [Enter]
How old are you? 25 [Enter]
What is your annual income? 60000 [Enter]
Your name is Janice Smith, you are 25 years old,
and you make $60000.00 per year.
```

Checkpoint

10.11 Write a short description of each of the following functions:

```
atoi        stoi
atol        stol
atof        stof
itoa        stod
```

10.12 Write a statement that will convert the string "10" to an integer and store the result in the variable `num`.

10.13 Write a statement that will convert the string "100000" to a `long` and store the result in the variable `num`.

10.14 Write a statement that will convert the string "7.2389" to a `double` and store the result in the variable `num`.

10.15 Write a statement that will convert the integer 127 to a string, and assign the result to a `string` object named `str`.

10.6 Focus on Software Engineering: Writing Your Own C-String-Handling Functions

CONCEPT: **You can design your own specialized functions for manipulating strings.**

VideoNote
Writing a C-String-Handling Function

By being able to pass arrays as arguments, you can write your own functions for processing C-strings. For example, Program 10-12 uses a function to copy a C-string from one array to another.

Program 10-12

```
1  // This program uses a function to copy a C-string into an array.
2  #include <iostream>
3  using namespace std;
4
5  void stringCopy(char [], char []); // Function prototype
6
7  int main()
8  {
9      const int LENGTH = 30;   // Size of the arrays
10     char first[LENGTH];      // To hold the user's input
11     char second[LENGTH];     // To hold the copy
12
13     // Get a string from the user and store in first.
14     cout << "Enter a string with no more than "
15          << (LENGTH - 1) << " characters:\n";
16     cin.getline(first, LENGTH);
17
```

(program continues)

Program 10-12 *(continued)*

```
18       // Copy the contents of first to second.
19       stringCopy(first, second);
20
21       // Display the copy.
22       cout << "The string you entered is:\n" << second << endl;
23       return 0;
24   }
25
26   //*********************************************************
27   // Definition of the stringCopy function.                 *
28   // This function copies the C-string in string1 to string2.*
29   //*********************************************************
30
31   void stringCopy(char string1[], char string2[])
32   {
33       int index = 0;  // Loop counter
34
35       // Step through string1, copying each element to
36       // string2. Stop when the null character is encountered.
37       while (string1[index] != '\0')
38       {
39           string2[index] = string1[index];
40           index++;
41       }
42
43       // Place a null character in string2.
44       string2[index] = '\0';
45   }
```

Program Output with Example Input Shown in Bold

```
Enter a string with no more than 29 characters:
Thank goodness it's Friday! Enter
The string you entered is:
Thank goodness it's Friday!
```

Notice the function `stringCopy` does not accept an argument indicating the size of the arrays. It simply copies the characters from `string1` into `string2` until it encounters a null terminator in `string1`. When the null terminator is found, the loop has reached the end of the C-string. The last statement in the function assigns a null terminator (the `'\0'` character) to the end of `string2`, so it is properly terminated.



WARNING! Because the `stringCopy` function doesn't know the size of the second array, it's the programmer's responsibility to make sure the second array is large enough to hold the string in the first array.

Program 10-13 uses another C-string-handling function: `nameSlice`. The program asks the user to enter his or her first and last names, separated by a space. The function searches the string for the space and replaces it with a null terminator. In effect, this "cuts" the last name off of the string.

Program 10-13

```
1  // This program uses the function nameSlice to cut the last
2  // name off of a string that contains the user's first and
3  // last names.
4  #include <iostream>
5  using namespace std;
6
7  void nameSlice(char []);  // Function prototype
8
9  int main()
10 {
11     const int SIZE = 41; // Array size
12     char name[SIZE];     // To hold the user's name
13
14     cout << "Enter your first and last names, separated ";
15     cout << "by a space:\n";
16     cin.getline(name, SIZE);
17     nameSlice(name);
18     cout << "Your first name is: " << name << endl;
19     return 0;
20 }
21
22 //*************************************************************
23 // Definition of function nameSlice. This function accepts a   *
24 // character array as its argument. It scans the array looking *
25 // for a space. When it finds one, it replaces it with a null  *
26 // terminator.                                                 *
27 //*************************************************************
28
29 void nameSlice(char userName[])
30 {
31     int count = 0; // Loop counter
32
33     // Locate the first space, or the null terminator if there
34     // are no spaces.
35     while (userName[count] != ' ' && userName[count] != '\0')
36         count++;
37
38     // If a space was found, replace it with a null terminator.
39     if (userName[count] == ' ')
40         userName[count] = '\0';
41 }
```

Program Output with Example Input Shown in Bold

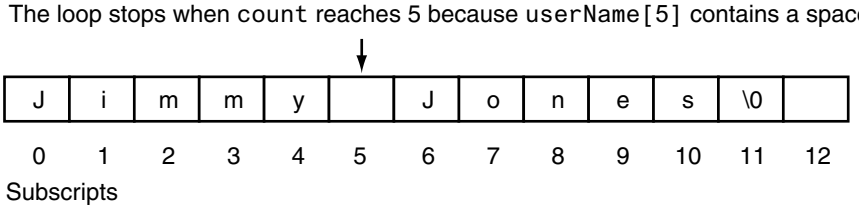
```
Enter your first and last names, separated by a space:
Jimmy Jones [Enter]
Your first name is: Jimmy
```

The following loop in lines 35 and 36 starts at the first character in the array and scans the string searching for either a space or a null terminator:

```
while (userName[count] != ' ' && userName[count] != '\0')
   count++;
```

If the character in `userName[count]` isn't a space or the null terminator, `count` is incremented, and the next character is examined. With the example input "`Jimmy Jones`," the loop finds the space separating "`Jimmy`" and "`Jones`" at `userName[5]`. When the loop stops, `count` is set to 5. This is illustrated in Figure 10-3.

Figure 10-3 Scanning for a space in a string





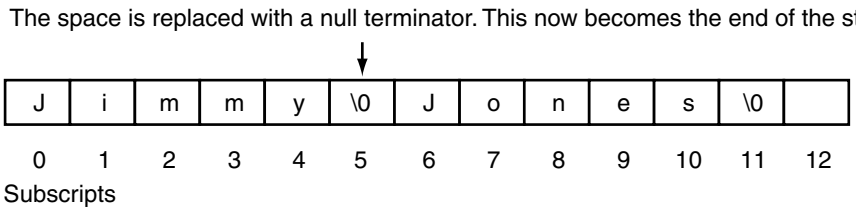
NOTE: The loop will also stop if it encounters a null terminator. This is so it will not go beyond the boundary of the array if the user didn't enter a space.

Once the loop has finished, `userName[count]` will either contain a space or a null terminator. If it contains a space, the following `if` statement (in lines 39 and 40) replaces it with a null terminator:

```
if (userName[count] == ' ')
   userName[count] = '\0';
```

This is illustrated in Figure 10-4.

Figure 10-4 Replacing a space with a null terminator



The new null terminator now becomes the end of the string.

Using Pointers to Pass C-String Arguments

Pointers are extremely useful for writing functions that process C-strings. If the starting address of a string is passed into a pointer parameter variable, it can be assumed that all the characters, from that address up to the byte that holds the null terminator, are part of the string. (It isn't necessary to know the length of the array that holds the string.)

Program 10-14 demonstrates a function, `countChars`, that uses a pointer to count the number of times a specific character appears in a C-string.

Program 10-14

```
1  // This program demonstrates a function, countChars, that counts
2  // the number of times a specific character appears in a string.
3  #include <iostream>
```

```
 4  using namespace std;
 5
 6  int countChars(char *, char); // Function prototype
 7
 8  int main()
 9  {
10      const int SIZE = 51;     // Array size
11      char userString[SIZE];   // To hold a string
12      char letter;             // The character to count
13
14      // Get a string from the user.
15      cout << "Enter a string (up to 50 characters): ";
16      cin.getline(userString, SIZE);
17
18      // Choose a character whose occurrences within the string will be counted.
19      cout << "Enter a character and I will tell you how many\n";
20      cout << "times it appears in the string: ";
21      cin >> letter;
22
23      // Display the number of times the character appears.
24      cout << letter << " appears ";
25      cout << countChars(userString, letter) << " times.\n";
26      return 0;
27  }
28
29  //*************************************************************
30  // Definition of countChars. The parameter strPtr is a pointer    *
31  // that points to a string. The parameter Ch is a character that  *
32  // the function searches for in the string. The function returns  *
33  // the number of times the character appears in the string.       *
34  //*************************************************************
35
36  int countChars(char *strPtr, char ch)
37  {
38      int times = 0;  // Number of times ch appears in the string
39
40      // Step through the string counting occurrences of ch.
41      while (*strPtr != '\0')
42      {
43          if (*strPtr == ch)       // If the current character equals ch...
44              times++;             // ... increment the counter
45          strPtr++;                // Go to the next char in the string.
46      }
47
48      return times;
49  }
```

Program Output with Example Input Shown in Bold

```
Enter a string (up to 50 characters): Starting Out with C++ [Enter]
Enter a character and I will tell you how many
times it appears in the string: t [Enter]
t appears 4 times.
```

In the function countChars, strPtr points to the C-string that is to be searched, and ch contains the character for which to look. The while loop in lines 41 through 46 repeats as long as the character that strPtr points to is not the null terminator:

```
while (*strPtr != '\0')
```

Inside the loop, the if statement in line 43 compares the character that strPtr points to with the character in ch:

```
if (*strPtr == ch)
```

If the two are equal, the variable times is incremented in line 44. (times keeps a running total of the number of times the character appears.) The last statement in the loop is

```
strPtr++;
```

This statement increments the address in strPtr. This causes strPtr to point to the next character in the string. Then, the loop starts over. When strPtr finally reaches the null terminator, the loop terminates, and the function returns the value in times.

For another example, see the String Manipulation Case Study, available for download from the Computer Science Portal at www.pearsonglobaleditions.com/gaddis.



Checkpoint

10.16 What is the output of the following program?

```
#include <iostream>
using namespace std;

// Function Prototype
void mess(char []);

int main()
{
   char stuff[] = "Tom Talbert Tried Trains";
   cout << stuff << endl;
   mess(stuff);
   cout << stuff << endl;
   return 0;
}

// Definition of function mess
void mess(char str[])
{
   int step = 0;

   while (str[step] != '\0')
   {
       if (str[step] == 'T')
          str[step] = 'D';
       step++;
   }
}
```

10.7 More about the C++ string Class

CONCEPT: **Standard C++ provides a special data type for storing and working with strings.**



VideoNote
More about the string Class

The `string` class is an abstract data type. This means it is not a built-in, primitive data type like `int` or `char`. Instead, it is a programmer-defined data type that accompanies the C++ language. It provides many capabilities that make storing and working with strings easy and intuitive.

Using the string Class

The first step in using the `string` class is to `#include` the `<string>` header file. This is accomplished with the following preprocessor directive:

```
#include <string>
```

Now you are ready to define a `string` object. Defining a `string` object is similar to defining a variable of a primitive type. For example, the following statement defines a `string` object named `movieTitle`:

```
string movieTitle;
```

You assign a string value to the `movieTitle` object with the assignment operator, as shown in the following statement:

```
movieTitle = "Wheels of Fury";
```

The contents of `movieTitle` are displayed on the screen with the `cout` object, as shown in the next statement:

```
cout << "My favorite movie is " << movieTitle << endl;
```

Program 10-15 is a complete program that demonstrates the statements shown above.

Program 10-15

```
1  // This program demonstrates the string class.
2  #include <iostream>
3  #include <string>     // Required for the string class.
4  using namespace std;
5
6  int main()
7  {
8      string movieTitle;
9
10     movieTitle = "Wheels of Fury";
11     cout << "My favorite movie is " << movieTitle << endl;
12     return 0;
13 }
```

Program Output

```
My favorite movie is Wheels of Fury
```

As you can see, working with `string` objects is similar to working with variables of other types. For example, Program 10-16 demonstrates how you can use `cin` to read a value from the keyboard into a `string` object.

Program 10-16

```
1  // This program demonstrates how cin can read a string into
2  // a string class object.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main()
8  {
9      string name;
10
11     cout << "What is your name? ";
12     cin >> name;
13     cout << "Good morning " << name << endl;
14     return 0;
15 }
```

Program Output with Example Input Shown in Bold

```
What is your name? Peggy [Enter]
Good morning Peggy
```

Reading a Line of Input into a `string` Object

If you want to read a line of input (with spaces) into a `string` object, use the `getline()` function. Here is an example:

```
string name;
cout << "What is your name? ";
getline(cin, name);
```

The `getline()` function's first argument is the name of a stream object from which you wish to read the input. The function call above passes the `cin` object to `getline()`, so the function reads a line of input from the keyboard. The second argument is the name of a `string` object. This is where `getline()` stores the input that it reads.

Comparing and Sorting `string` Objects

There is no need to use a function such as `strcmp` to compare `string` objects. You may use the `<`, `>`, `<=`, `>=`, `==`, and `!=` relational operators. For example, assume the following definitions exist in a program:

```
string set1 = "ABC";
string set2 = "XYZ";
```

The object `set1` is considered less than the object `set2` because the characters "ABC" alphabetically precede the characters "XYZ." So, the following `if` statement will cause the message "set1 is less than set2" to be displayed on the screen:

```
if (set1 < set2)
   cout << "set1 is less than set2.\n";
```

Relational operators perform comparisons on `string` objects in a fashion similar to the way the `strcmp` function compares C-strings. One by one, each character in the first operand is compared with the character in the corresponding position in the second operand. If all the characters in both strings match, the two strings are equal. Other relationships can be determined if two characters in corresponding positions do not match. The first operand is less than the second operand if the mismatched character in the first operand is less than its counterpart in the second operand. Likewise, the first operand is greater than the second operand if the mismatched character in the first operand is greater than its counterpart in the second operand.

For example, assume a program has the following definitions:

```
string name1 = "Mary";
string name2 = "Mark";
```

The value in `name1`, "Mary," is greater than the value in `name2`, "Mark." This is because the "y" in "Mary" has a greater ASCII value than the "k" in "Mark."

`string` objects can also be compared to C-strings with relational operators. Assuming `str` is a `string` object, all of the following are valid relational expressions:

```
str > "Joseph"
"Kimberly" < str
str == "William"
```

Program 10-17 demonstrates `string` objects and relational operators.

Program 10-17

```
1  // This program uses the == operator to compare a string entered
2  // by the user with the valid part numbers.
3  #include <iostream>
4  #include <iomanip>
5  #include <string>
6  using namespace std;
7
8  int main()
9  {
10    const double APRICE = 249.0;    // Price for part A
11    const double BPRICE = 199.0;    // Price for part B
12    string partNum;                 // Part mumber
13
14    cout << "The headphone part numbers are:\n";
15    cout << "\tNoise canceling, part number S147-29A\n";
16    cout << "\tWireless, part number S147-29B\n";
17    cout << "Enter the part number of the desired headphones: ";
18    cin >> partNum;
19    cout << fixed << showpoint << setprecision(2);
20
21    if (partNum == "S147-29A")
22       cout << "The price is $" << APRICE << endl;
23    else if (partNum == "S147-29B")
24       cout << "The price is $" << BPRICE << endl;
25    else
26       cout << partNum << " is not a valid part number.\n";
```

(program continues)

Program 10-17 *(continued)*

```
27      return 0;
28  }
```

Program Output with Example Input Shown in Bold

```
The headphone part numbers are:
        Noise canceling, part number S147-29A
        Wireless, part number S147-29B
Enter the part number of the desired headphones: S147-29B [Enter]
The price is $199.00
```

You may also use relational operators to sort `string` objects. Program 10-18 demonstrates this.

Program 10-18

```
 1  // This program uses relational operators to alphabetically
 2  // sort two strings entered by the user.
 3  #include <iostream>
 4  #include <string>
 5  using namespace std;
 6
 7  int main ()
 8  {
 9      string name1, name2;
10
11      // Get a name.
12      cout << "Enter a name (last name first): ";
13      getline(cin, name1);
14
15      // Get another name.
16      cout << "Enter another name: ";
17      getline(cin, name2);
18
19      // Display them in alphabetical order.
20      cout << "Here are the names sorted alphabetically:\n";
21      if (name1 < name2)
22          cout << name1 << endl << name2 << endl;
23      else if (name1 > name2)
24          cout << name2 << endl << name1 << endl;
25      else
26          cout << "You entered the same name twice!\n";
27      return 0;
28  }
```

Program Output with Example Input Shown in Bold

```
Enter a name (last name first): Smith, Richard [Enter]
Enter another name: Jones, John [Enter]
Here are the names sorted alphabetically:
Jones, John
Smith, Richard
```

Other Ways to Define string Objects

There are a variety of ways to initialize a string object when you define it. Table 10-7 shows several example definitions and describes each. Program 10-19 demonstrates a string object initialized with the string "William Smith."

Program 10-19

```
1  // This program initializes a string object.
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  int main()
7  {
8       string greeting;
9       string name("William Smith");
10
11      greeting = "Hello ";
12      cout << greeting << name << endl;
13      return 0;
14 }
```

Program Output

```
Hello William Smith
```

Table 10-7 Examples of string Object Definitions

Definition	Description
`string address;`	Defines an empty string object named address.
`string name("William Smith");`	Defines a string object named name, initialized with "William Smith."
`string person1(person2);`	Defines a string object named person1, which is a copy of person2. person2 may be either a string object or character array.
`string str1(str2, 5);`	Defines a string object named str1, which is initialized to the first five characters in the character array str2.
`string lineFull('z', 10);`	Defines a string object named lineFull initialized with 10 'z' characters.
`string firstName(fullName, 0, 7);`	Defines a string object named firstName, initialized with a substring of the string fullName. The substring is seven characters long, beginning at position 0.

Notice in Program 10-19 the use of the = operator to assign a value to the string object (line 11). The string class supports several operators, which are described in Table 10-8.

Table 10-8 string Class Operators

Supported Operator	Description
>>	Extracts characters from a stream and inserts them into the string. Characters are copied until a whitespace or the end of the string is encountered.
<<	Inserts the string into a stream.
=	Assigns the string on the right to the string object on the left.
+=	Appends a copy of the string on the right to the string object on the left.
+	Returns a string that is the concatenation of the two string operands.
[]	Implements array-subscript notation, as in name[x]. A reference to the character in the x position is returned.
Relational Operators	Each of the relational operators is implemented: < > <= >= == ! =

Program 10-20 demonstrates some of the string operators.

Program 10-20

```
1  // This program demonstrates the C++ string class.
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  int main ()
7  {
8       // Define three string objects.
9       string str1, str2, str3;
10
11      // Assign values to all three.
12      str1 = "ABC";
13      str2 = "DEF";
14      str3 = str1 + str2;
15
16      // Display all three.
17      cout << str1 << endl;
18      cout << str2 << endl;
19      cout << str3 << endl;
20
21      // Concatenate a string onto str3 and display it.
22      str3 += "GHI";
23      cout << str3 << endl;
24      return 0;
25 }
```

Program Output

```
ABC
DEF
ABCDEF
ABCDEFGHI
```

Using string Class Member Functions

The string class also has member functions. For example, the length member function returns the length of the string stored in the object. The value is returned as an unsigned integer.

Assume the following string object definition exists in a program:

```
string town = "Charleston";
```

The following statement in the same program would assign the value 10 to the variable x.

```
x = town.length();
```

Program 10-21 further demonstrates the length member function.

Program 10-21

```
1  // This program demonstrates a string
2  // object's length member function.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main ()
8  {
9       string town;
10
11      cout << "Where do you live? ";
12      cin >> town;
13      cout << "Your town's name has " << town.length() ;
14      cout << " characters\n";
15      return 0;
16 }
```

Program Output with Example Input Shown in Bold

```
Where do you live? Jacksonville Enter
Your town's name has 12 characters
```

The size function also returns the length of the string. It is demonstrated in the for loop in Program 10-22.

Program 10-22

```
1  // This program demonstrates the C++ string class.
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  int main()
7  {
8       // Define three string objects.
9       string str1, str2, str3;
10
11      // Assign values to all three.
```

(program continues)

Program 10-22 *(continued)*

```
12      str1 = "ABC";
13      str2 = "DEF";
14      str3 = str1 + str2;
15
16      // Use subscripts to display str3 one character
17      // at a time.
18      for (int x = 0; x < str3.size(); x++)
19          cout << str3[x];
20      cout << endl;
21
22      // Compare str1 with str2.
23      if (str1 < str2)
24          cout << "str1 is less than str2\n";
25      else
26          cout << "str1 is not less than str2\n";
27      return 0;
28  }
```

Program Output

```
ABCDEF
str1 is less than str2
```

Table 10-9 lists many of the `string` class member functions and their overloaded variations. In the examples, assume `mystring` is the name of a `string` object.

Table 10-9 `string` Class Member Functions

Member Function Example	Description
`mystring.append(n, 'z')`	Appends `n` copies of `'z'` to `mystring`.
`mystring.append(str)`	Appends `str` to `mystring`. `str` can be a `string` object or character array.
`mystring.append(str, n)`	The first `n` characters of the character array `str` are appended to `mystring`.
`mystring.append(str, x, n)`	`n` number of characters from `str`, starting at position `x`, are appended to `mystring`. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.assign(n, 'z')`	Assigns `n` copies of `'z'` to `mystring`.
`mystring.assign(str)`	Assigns `str` to `mystring`. `str` can be a `string` object or character array.
`mystring.assign(str, n)`	The first `n` characters of the character array `str` are assigned to `mystring`.
`mystring.assign(str, x, n)`	`n` number of characters from `str`, starting at position `x`, are assigned to `mystring`. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.at(x)`	Returns the character at position `x` in the `string`.
`mystring.back()`	Returns the last character in the `string`. (This member function was introduced in C++ 11.)

Table 10-9 *(continued)*

Member Function Example	Description
`mystring.begin()`	Returns an iterator pointing to the first character in the `string`. (For more information on iterators, see Chapter 16.)
`mystring.c_str()`	Converts the contents of `mystring` to a C-string, and returns a pointer to the C-string.
`mystring.capacity()`	Returns the size of the storage allocated for the `string`.
`mystring.clear()`	Clears the `string` by deleting all the characters stored in it.
`mystring.compare(str)`	Performs a comparison like the `strcmp` function (see Chapter 4), with the same return values. `str` can be a `string` object or a character array.
`mystring.compare(x, n, str)`	Compares `mystring` and `str`, starting at position `x`, and continuing for `n` characters. The return value is like `strcmp`. `str` can be a `string` object or character array.
`mystring.copy(str, x, n)`	Copies the character array `str` to `mystring`, beginning at position `x`, for `n` characters. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.empty()`	Returns `true` if `mystring` is empty.
`mystring.end()`	Returns an iterator pointing to the last character of the string in `mystring`. (For more information on iterators, see Chapter 17.)
`mystring.erase(x, n)`	Erases `n` characters from `mystring`, beginning at position `x`.
`mystring.find(str, x)`	Returns the first position at or beyond position `x` where the string `str` is found in `mystring`. `str` may be either a `string` object or a character array.
`mystring.find('z', x)`	Returns the first position at or beyond position `x` where `'z'` is found in `mystring`. If `'z'` is not found, the function returns the special value `string::npos`.
11 `mystring.front()`	Returns the first character in the `string`. (This member function was introduced in C++ 11.)
`mystring.insert(x, n, 'z')`	Inserts `'z'` `n` times into `mystring` at position `x`.
`mystring.insert(x, str)`	Inserts a copy of `str` into `mystring`, beginning at position `x`. `str` may be either a `string` object or a character array.
`mystring.length()`	Returns the length of the string in `mystring`.
`mystring.replace(x, n, str)`	Replaces the `n` characters in `mystring` beginning at position `x` with the characters in string object `str`.
`mystring.resize(n, 'z')`	Changes the size of the allocation in `mystring` to `n`. If `n` is less than the current size of the string, the string is truncated to `n` characters. If `n` is greater, the string is expanded and `'z'` is appended at the end enough times to fill the new spaces.
`mystring.size()`	Returns the length of the string in `mystring`.
`mystring.substr(x, n)`	Returns a copy of a substring. The substring is `n` characters long and begins at position `x` of `mystring`.
`mystring.swap(str)`	Swaps the contents of `mystring` with `str`.



In the Spotlight:

String Tokenizing

Sometimes a string will contain a series of words or other items of data separated by spaces or other characters. For example, look at the following string:

```
"peach raspberry strawberry vanilla"
```

This string contains the following four items of data: `peach`, `raspberry`, `strawberry`, and `vanilla`. In programming terms, items such as these are known as *tokens*. Notice a space appears between the items. The character that separates tokens is known as a *delimiter*. Here is another example:

```
"17;92;81;12;46;5"
```

This string contains the following tokens: 17, 92, 81, 12, 46, and 5. Notice a semicolon appears between each item. In this example, the semicolon is used as a delimiter. Some programming problems require you to read a string that contains a list of items then extract all of the tokens from the string for processing. For example, look at the following string that contains a date:

```
"3–22–2018"
```

The tokens in this string are 3, 22, and 2018, and the delimiter is the hyphen character. Perhaps a program needs to extract the month, day, and year from such a string. Another example is an operating system pathname, such as the following:

```
/home/rsullivan/data
```

The tokens in this string are `home`, `rsullivan`, and `data`, and the delimiter is the '`/`' character. Perhaps a program needs to extract all of the directory names from such a pathname. The process of breaking a string into tokens is known as *tokenizing*, or *splitting* a string.

The following function, `split`, shows an example of how we can split a string into tokens. The function has three parameters:

- `s`—the string that we want to split into tokens
- `delim`—the character that is used as a delimiter
- `tokens`—a `vector` that will hold the tokens once they are extracted

```
1  void split(const string& s, char delim, vector<string>& tokens)
2  {
3     int tokenStart = 0;  // Starting position of the next token
4
5     // Find the first occurrence of the delimiter.
6     int delimPosition = s.find(delim);
7
8     // While we haven't run out of delimiters...
9     while (delimPosition != string::npos)
10    {
11       // Extract the token.
12       string tok = s.substr(tokenStart, delimPosition - tokenStart);
13
14       // Push the token onto the tokens vector.
15       tokens.push_back(tok);
```

```
16
17          // Move delimPosition to the next character position.
18          delimPosition++;
19
20          // Move tokenStart to delmiPosition.
21          tokenStart = delimPosition;
22
23          // Find the next occurrence of the delimiter.
24          delimPosition = s.find(delim, delimPosition);
25
26          // If no more delimiters, extract the last token.
27          if (delimPosition == string::npos)
28          {
29              // Extract the token.
30              string tok = s.substr(tokenStart, delimPosition - tokenStart);
31
32              // Push the token onto the vector.
33              tokens.push_back(tok);
34          }
35      }
36  }
```

Let's take a closer look at the code:

- The `tokenStart` variable defined in line 3 will be used to hold the starting position of the next token. This variable is initialized with 0, assuming the first token starts at position 0.
- In line 6, we call the `s.find()` function, passing `delim` as an argument. The function will return the position of the first occurrence of `delim` in `s`. That value is assigned to the `delimPosition` variable. Note if the `find()` member function does not find the specified delimiter, it will return the special constant `string::npos` (which is defined as –1).
- The `while` loop that begins in line 9 iterates as long as `delimPosition` is not equal to `string::npos`.
- Inside the `while` loop, line 12 extracts the substring beginning at `tokenStart`, and continuing up to the delimiter. The substring, which is a token, is assigned to the `tok` object.
- In line 15, we push the `tok` object to the back of the `tokens` vector.
- Now we are ready to find the next token. Line 18 increments `delimPosition`, and line 21 sets `tokenStart` to the same value as `delimPosition`.
- Line 24 calls the `s.find()` function, passing `delim` and `delimPosition` as arguments. The function will return the position of the next occurrence of `delim`, appearing at or after `delimPosition`. That value is assigned to the `delimPosition` variable. If the specified delimiter is not found, the `find()` member function will return the constant `string::npos`. When that happens in this statement, it means we are processing the last token in the string.
- The `if` statement that starts in line 27 determines whether `delimPosition` is equal to `string::npos`. As previously stated, if this is true, it means we are processing the last token. If that is the case, line 30 extracts the token and assigns it to `tok`, and line 33 pushes `tok` to the back of the `tokens` vector.

Program 10-23 demonstrates the `split` function.

Program 10-23

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Function prototype
void split(const string&, char, vector<string>&);

int main()
{
   // Strings to tokenize
   string str1 = "one two three four";
   string str2 = "10:20:30:40:50";
   string str3 = "a/b/c/d/e/f";

   // vector to hold the tokens.
   vector<string> tokens;

   // Tokenize str1, using ' ' as the delimiter.
   split(str1, ' ', tokens);
   for (auto e : tokens)
      cout << e << " ";
   cout << endl;
   tokens.clear();

   // Tokenize str2, using ':' as the delimiter.
   split(str2, ':', tokens);
   for (auto e : tokens)
      cout << e << " ";
   cout << endl;
   tokens.clear();

   // Tokenize str3, using '/' as the delimiter.
   split(str3, '/', tokens);
   for (auto e : tokens)
      cout << e << " ";
   cout << endl;
   return 0;
}

The split function is not shown here.
```

Program Output

```
one two three four
10 20 30 40 50
a b c d e f
```

10.8 Focus on Problem Solving and Program Design: A Case Study

As a programmer for the Home Software Company, you are asked to develop a function named `dollarFormat` that inserts commas and a $ sign at the appropriate locations in a `string` object containing an unformatted dollar amount. As an argument, the function should accept a reference to a `string` object. You may assume the `string` object contains a value such as 1084567.89. The function should modify the `string` object so it contains a formatted dollar amount, such as $1,084,567.89.

The code for the `dollarFormat` function is as follows:

```
void dollarFormat(string &currency)
{
   int dp;
   dp = currency.find('.');    // Find decimal point
   if (dp > 3)                 // Insert commas
   {
      for (int x = dp - 3; x > 0; x -= 3)
          currency.insert(x, ",");
   }
   currency.insert(0, "$");    // Insert dollar sign
}
```

The function defines an `int` variable named `dp`. This variable is used to hold the position of the unformatted number's decimal point. This is accomplished with the statement:

```
dp = currency.find('.');
```

The `string` class's `find` member function returns the position number in the string where the '.' character is found. An `if` statement determines if the number has more than three numbers preceding the decimal point:

```
if (dp > 3)
```

If the decimal point is at a position greater than 3, then the function inserts commas in the string with the following loop:

```
for (int x = dp - 3; x > 0; x -= 3)
    currency.insert(x, ",");
```

Finally, a $ symbol is inserted at position 0 (the first character in the `string`).

Program 10-24 demonstrates the function.

Program 10-24

```
1  // This program lets the user enter a number. The
2  // dollarFormat function formats the number as
3  // a dollar amount.
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  // Function prototype
```

(program continues)

Program 10-24 *(continued)*

```
 9  void dollarFormat(string &);
10
11  int main ()
12  {
13       string input;
14
15       // Get the dollar amount from the user.
16       cout << "Enter a dollar amount in the form nnnnn.nn : ";
17       cin >> input;
18       dollarFormat(input);
19       cout << "Here is the amount formatted:\n";
20       cout << input << endl;
21       return 0;
22  }
23
24  //*************************************************************
25  // Definition of the dollarFormat function. This function    *
26  // accepts a string reference object, which is assumed       *
27  // to hold a number with a decimal point. The function       *
28  // formats the number as a dollar amount with commas and     *
29  // a $ symbol.                                               *
30  //*************************************************************
31
32  void dollarFormat(string &currency)
33  {
34       int dp;
35
36       dp = currency.find('.');     // Find decimal point
37       if (dp > 3)                  // Insert commas
38       {
39           for (int x = dp - 3; x > 0; x -= 3)
40                currency.insert(x, ",");
41       }
42       currency.insert(0, "$");     // Insert dollar sign
43  }
```

Program Output with Example Input Shown in Bold

```
Enter a dollar amount in the form nnnnn.nn: 1084567.89 Enter
Here is the amount formatted:
$1,084,567.89
```

Review Questions and Exercises

Short Answer

1. What header file must you include in a program using character-testing functions such as `isalpha` and `isdigit`?
2. What header file must you include in a program using the character conversion functions `toupper` and `tolower`?

3. Assume c is a char variable. What value does c hold after each of the following statements executes?

Statement	Contents of c
`c = toupper('a');`	_____
`c = toupper('B');`	_____
`c = tolower('D');`	_____
`c = toupper('e');`	_____

4. Look at the following code. What value will be stored in s after the code executes?

```
char name[10];
int s;
strcpy(name, "Jimmy");
s = strlen(name);
```

5. What header file must you include in a program using string functions such as strlen and strcpy?
6. What header file must you include in a program using string/numeric conversion functions such as atoi and atof?
7. What header file must you include in a program using string class objects?
8. How do you compare string class objects?

Fill-in-the-Blank

9. Character-testing functions are prototyped in the header file _____.
10. The _____ function returns true if the character argument is a letter of the alphabet.
11. The _____ function returns true if the character argument is a digit.
12. _____, _____, _____ and _____ are whitespace characters.
13. The _____ function returns the uppercase equivalent of its character argument.
14. The _____ function returns the lowercase equivalent of its character argument.
15. The _____ file must be included in a program that uses character-testing functions.
16. A _____ is a sequence of characters terminated by a null character.
17. To _____ two strings means to append one string to the other.
18. The _____ function concatenates two strings.
19. The _____ function copies one string to another.
20. The _____ function searches for a string inside of another one.
21. Slash zero '\0' stands for the ASCII code _____.
22. The _____ function copies, at most, *n* number of characters from one string to another.
23. The _____ function returns the value of a string converted to an integer.
24. The _____ function returns the value of a string converted to a long integer.
25. The _____ function returns the value of a string converted to a float.
26. To use a string object, the header file _____ needs to be included.

Algorithm Workbench

27. The following `if` statement determines whether choice is equal to `'Y'` or `'y'`:

    ```
    if (choice == 'Y' || choice == 'y')
    ```

    Simplify this statement by using either the `toupper` or `tolower` function.

28. Assume that `mystr` is a `char` array holding a C-string. Write code that prints only those characters that are in lowercase.

29. Look at the following character arrays:

    ```
    char fromstr[10], tostr[10];
    ```

    Write code that copies characters from `fromstr` to `tostr` without using any string library functions.

30. Look at the following statements:

    ```
    char chars[20];
    string stringValue;
    ```

    Write code that swaps the first and last characters in the variable `chars` holding a C-string. Write another version for the variable `stringValue`.

31. Write a function that accepts a pointer to a C-string as its argument. The function should count the number of times the character 'w' occurs in the argument and return that number.

32. Assume `str1` and `str2` are string class objects. Write code that returns true if `str1` is a substring of `str2` or `str2` is a substring of `str1`.

True or False

33. T F Character-testing functions, such as `isupper`, accept strings as arguments and test each character in the string.

34. T F If `toupper`'s argument is already uppercase, it is returned as is, with no changes.

35. T F If `tolower`'s argument is already lowercase, it will be inadvertently converted to uppercase.

36. T F The character-testing functions return a nonzero number if the test condition passes.

37. T F If the starting address of a C-string is passed into a pointer parameter, it can be assumed that all the characters, from that address up to the byte that holds the null terminator, are part of the string.

38. T F The `strcat` function performs bound checking before appending one string at the end of another so that no overflow occurs while appending.

39. T F The expression (`!strcmp(firstString, secondString)`) returns true if the strings are unequal.

40. T F The `to_string` function converts only integers to strings.

41. T F The `strcpy` function performs no bounds checking on the first argument.

42. T F There is no difference between "847" and 847.

Find the Errors

Each of the following programs or program segments has errors. Find as many as you can.

43.
```
char str[] = "Hello";
str[0] = "Y";
str[5] = 'w';
```

44.
```
char str[] = "172";
int eight = str[0] + str[1];
char nine = atoi(str[1]) + atoi(str[2]);
```

45.
```
char string1[] = "Billy";
char string2[] = " Bob Jones";
strcat(string1, string2);
```

46.
```
string str3, str1= "A";
string str2 = "a";
       str3 = (str1++) + (++str2);
```

Programming Challenges

1. **String Concatenation**

   Write a function that accepts two C-string pointers as arguments and returns a pointer to another C-string. The function should concatenate the two strings passed as arguments separated by a blank space, and return a pointer to the concatenated string. Demonstrate the function using a program. Do not use any string library functions.



VideoNote
Solving the Backward String Problem

2. **Backward String**

   Write a function that accepts a pointer to a C-string as an argument and displays its contents backward. For instance, if the string argument is `"Gravity"` the function should display `"ytivarG"`. Demonstrate the function in a program that asks the user to input a string then passes it to the function.

3. **Word Counter**

   Write a function that accepts a pointer to a C-string as an argument and returns the number of words contained in the string. For instance, if the string argument is "Four score and seven years ago" the function should return the number 6. Demonstrate the function in a program that asks the user to input a string then passes it to the function. The number of words in the string should be displayed on the screen. *Optional Exercise:* Write an overloaded version of this function that accepts a `string` class object as its argument.

4. **Average Number of Letters**

   Modify the program you wrote for Programming Challenge 3 (Word Counter), so it also displays the average number of letters in each word.

5. **Sentence Capitalizer**

   Write a function that accepts a pointer to a C-string as an argument and capitalizes the first character of each sentence in the string. For instance, if the string argument is `"hello. my name is Joe. what is your name?"` the function should manipulate the string so that it contains `"Hello. My name is Joe. What is your name?"`

Demonstrate the function in a program that asks the user to input a string then passes it to the function. The modified string should be displayed on the screen. *Optional Exercise:* Write an overloaded version of this function that accepts a `string` class object as its argument.

6. Vowels and Consonants

Write a function that accepts a pointer to a C-string as its argument. The function should count the number of vowels appearing in the string and return that number.

Write another function that accepts a pointer to a C-string as its argument. This function should count the number of consonants appearing in the string and return that number.

Demonstrate these two functions in a program that performs the following steps:

1. The user is asked to enter a string.
2. The program displays the following menu:
   A) Count the number of vowels in the string
   B) Count the number of consonants in the string
   C) Count both the vowels and consonants in the string
   D) Enter another string
   E) Exit the program
3. The program performs the operation selected by the user and repeats until the user selects E to exit the program.

7. Name Arranger

Write a program that asks for the user's first, middle, and last names. The names should be stored in three different character arrays. The program should then store, in a fourth array, the name arranged in the following manner: the last name followed by a comma and a space, followed by the first name and a space, followed by the middle name. For example, if the user entered `"Carol Lynn Smith"`, it should store `"Smith, Carol Lynn"` in the fourth array. Display the contents of the fourth array on the screen.

8. Guessing Game

Write a function for a word guessing game that takes two strings `current` and `correct`, and a single character `guess`. The string `current` should contain underscores (_) to indicate missing letters. The function should replace the underscores with the supplied character `guess` at the positions matching in `correct` and return the number of times the character can be found in `correct`. If the character supplied is not found, return 0.

For example, when `current` is `"p__g__mm__g"`, `correct` is `"programming"`, and `guess` equals `'r'`, the function should change `current` to `"pr_gr_mm__g"` and return 2.

9. Abbreviation of Name

Write a program that asks the user for his/her full name. The program should display his/her initials. For example, if the name is "Paul Mathews", the program will display "P. M.".

Input Validation: Accept only alphabets and blanks in name.

10. `replaceSubstring` Function

Write a function named `replaceSubstring`. The function should accept three C-string or `string` object arguments. Let's call them *`string1`*, *`string2`*, and *`string3`*. It should search *`string1`* for all occurrences of *`string2`*. When it finds an occurrence of

string2, it should replace it with *string3*. For example, suppose the three arguments have the following values:

`string1`:	"the dog jumped over the fence"
`string2`:	"the"
`string3`:	"that"

With these three arguments, the function would return a `string` object with the value "that dog jumped over that fence." Demonstrate the function in a complete program.

11. Case Manipulator

Write a program with three functions: `upper`, `lower`, and `reverse`. The `upper` function should accept a pointer to a C-string as an argument. It should step through each character in the string, converting it to uppercase. The `lower` function, too, should accept a pointer to a C-string as an argument. It should step through each character in the string, converting it to lowercase. Like `upper` and `lower`, `reverse` should also accept a pointer to a string. As it steps through the string, it should test each character to determine whether it is uppercase or lowercase. If a character is uppercase, it should be converted to lowercase. Likewise, if a character is lowercase, it should be converted to uppercase.

Test the functions by asking for a string in function `main`, then passing it to them in the following order: `reverse`, `lower`, and `upper`.

12. Validity of Filenames

A certain operating system specifies the following restrictions for naming files:

- Filenames must be, at the most, of eight characters.
- Filenames may have alphabets, digits, or the characters "_" and "–".
- Filenames should begin with an alphabet.
- Filenames must have a three-character extension name having only alphabets.

Write a function that accepts a filename and its extension name as arguments. It should return 1 if the filename is valid and 0 if it is not. Demonstrate the function in a program.

13. Time Conversion

Write a function that takes a C-string holding a time in the format hh:mm, and prints the time in AM/PM notation, with AM or PM at the end, and hh less than or equal to 12.

14. Word Separator

Write a program that accepts as input a sentence in which all of the words are run together, but the first character of each word is uppercase. Convert the sentence to a string in which the words are separated by spaces and only the first word starts with an uppercase letter. For example, the string "StopAndSmellTheRoses." would be converted to "Stop and smell the roses."

15. Character Analysis

If you have downloaded this book's source code, you will find a file named text.txt in the Chapter 10 folder. Write a program that reads the file's contents and determines the following:

- The number of uppercase letters in the file
- The number of lowercase letters in the file
- The number of digits in the file

16. Pig Latin

Write a program that reads a sentence as input and converts each word to "Pig Latin." In one version, to convert a word to Pig Latin, you remove the first letter and place that letter at the end of the word. Then you append the string "ay" to the word. Here is an example:

English: I SLEPT MOST OF THE NIGHT
Pig Latin: IAY LEPTSAY OSTMAY FOAY HETAY IGHTNAY

17. Morse Code Converter

Morse code is a code where each letter of the English alphabet, each digit, and various punctuation characters are represented by a series of dots and dashes. Table 10-10 shows part of the code.

Write a program that asks the user to enter a string then converts that string to Morse code.

Table 10-10 Morse Code

Character	Code	Character	Code	Character	Code	Character	Code
space	*space*	6	-....	G	--.	Q	--.-
comma	--..--	7	--...	H	....	R	.-.
period	.-.-.-	8	---..	I	..	S	...
question mark	..--..	9	----.	J	.---	T	-
0	-----	A	.-	K	-.-	U	..-
1	.----	B	-...	L	.-..	V	...-
2	..---	C	-.-.	M	--	W	.--
3	...--	D	-..	N	-.	X	-..-
4	....-	E	.	O	---	Y	-.--
5	.....	F	..-.	P	.--.	Z	--..

18. Phone Number List

Write a program that has an array of at least 10 string objects that hold people's names and phone numbers. You may make up your own strings, or use the following:

```
"Alejandra Cruz, 555-1223"
"Joe Looney, 555-0097"
"Geri Palmer, 555-8787"
"Li Chen, 555-1212"
"Holly Gaddis, 555-8878"
"Sam Wiggins, 555-0998"
"Bob Kain, 555-8712"
"Tim Haynes, 555-7676"
"Warren Gaddis, 555-9037"
"Jean James, 555-4939"
"Ron Palmer, 555-2783"
```

The program should ask the user to enter a name or partial name to search for in the array. Any entries in the array that match the string entered should be displayed. For example, if the user enters `"Palmer"` the program should display the following names from the list:

```
Geri Palmer, 555-8787
Ron Palmer, 555-2783
```

19. Check Writer

Write a program that displays a simulated paycheck. The program should ask the user to enter the date, the payee's name, and the amount of the check (up to $10,000). It should then display a simulated check with the dollar amount spelled out, as shown here:

```
                                                Date: 11/24/2018

Pay to the Order of: John Phillips                      $1920.85

One thousand nine hundred twenty and 85 cents
```

Be sure to format the numeric value of the check in fixed-point notation with two decimal places of precision. Be sure the decimal place always displays, even when the number is zero or has no fractional part. Use either C-strings or `string` class objects in this program.

Input Validation: Do not accept negative dollar amounts, or amounts over $10,000.

20. Format Checker

Many strings have to be formatted according to a set of rules. Phone numbers, for example, depending on the country have a specified length and fixed positions for slashes or dashes. Car license numbers may first have a specific number of characters, a dash, and finally another set of digits; emails should contain a @; and software licenses and serial numbers often consist of several groups of either numbers or characters.

Write a function that verifies whether a given string (an object of the string class) conforms to a given format. The function returns a Boolean: true when the string conforms and false when it doesn't conform.

The format is defined using two lists:

- A list of type `string`
- A list of type `int`

The two lists must be interpreted in pairs. The `string` represents the characters that are allowed, whereas the `int` represents the number of times that set of characters should occur.

For example, if the `string` list contains "ABC" and "12345" and the `int` list contains 3 and 2, then the string should contain A, B, or C thrice followed twice by 1, 2, 3, 4, or 5.